

Spec-Driven Development: An Implementation Cookbook

- Spec-Driven Development: An Implementation Cookbook
 - How to use this cookbook
- PART I — FOUNDATIONS
 - 1. The problem SDD solves
 - 2. The specification spectrum
 - 3. SDD versus the SRS, HLD and LLD you already write
 - 4. The core loop: Specify → Plan → Implement → Validate
 - The net effect the paper claims: agents move from reactive tools to proactive collaborators. The net effect you should plan for: **your agent output quality is now a function of your spec quality**, which makes spec review a first-class engineering activity with the same rigor as code review.
- PART II — THE ARTIFACT MODEL
 - 5. The spec stack
 - 6. Recipe: authoring a specification
 - 7. The traceability spine
- PART III — SDD ACROSS THE SDLC
 - 8. Stage 1 — Requirements
 - 9. Stage 2 — Design and planning
 - 10. Stage 3 — Task decomposition
 - 11. Stage 4 — Implementation
 - 12. Stage 5 — Integration, CI/CD and release
 - 13. Stage 6 — Maintenance, change and brownfield adoption
- PART IV — SDD ACROSS THE STLC
 - 14. STLC Phase 1 — Requirement analysis becomes spec testability review
 - 15. STLC Phase 2 — Test planning as level allocation
 - 16. STLC Phase 3 — Test design as derivation
 - 17. STLC Phase 4 — Environment and test data from the spec
 - 18. STLC Phase 5 — Execution
 - 19. STLC Phase 6 — Closure, defect arbitration and reporting
 - Baseline all ten **before** the pilot starts. Without a baseline you will be unable to defend the programme at the first budget review, and "it feels better" is not a defence.
- PART V — TOOLING AND REFERENCE PIPELINES
 - 20. The tool landscape
 - 21. Reference pipelines
- PART VI — EXECUTION: ROLLOUT, GOVERNANCE, PITFALLS
 - 22. The 90-day implementation plan
 - 23. Roles and the redefinition of work
 - 24. Pitfall register
 - 25. Governance
 - 26. Closing: what to hold onto
 - The question this discipline reframes, in the paper's closing words paraphrased: the developer's central question stops being "what code should I write?" and becomes "what specification should I provide?" Everything in Parts III and IV is machinery for answering the second question repeatably.
- APPENDIX A — TEMPLATES
 - A.1 spec.md
 - A.2 plan.md
 - A.3 tasks.md
 - A.4 test-spec.md
 - A.5 ADR
 - A.6 Defect record extension
- APPENDIX B — AGENT PROMPTS
 - B.1 Draft a spec from intent (self-spec pattern)
 - B.2 Adversarial ambiguity review
 - B.3 Derive acceptance scenarios
 - B.4 Derive property tests from business rules
 - B.5 Implement a task
 - B.6 Extract a spec from legacy code
 - B.7 Drift audit
 - B.8 Spec review for agent-readiness
- APPENDIX C — CI/CD WIRING
 - C.1 The gate ladder
 - C.2 Spec-drift labelling
 - C.3 Branch protection for spec files
- APPENDIX D — CHECKLISTS AND RUBRICS
 - D.1 Spec quality rubric
 - D.2 Spec review checklist (QE)
 - D.3 Agent-output review checklist
 - D.4 Release readiness checklist
- APPENDIX E — RTM GENERATION
 - E.1 Output shape

- E.2 Extractor sketch
- APPENDIX F — GLOSSARY AND SOURCES
 - F.1 Glossary
 - F.2 Primary source
 - F.3 Works referenced by the source paper, useful for implementation

Spec-Driven Development: An Implementation Cookbook

Plan, Process, Execution and Implementation across the SDLC and STLC

Grounded in: Deepak Babu Piskala, "*Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants*," Technical Report, arXiv:2602.00180v1 [cs.SE], 30 January 2026.

Audience: Engineering leads, QE/QA architects, test managers, platform teams and delivery managers who are introducing AI coding agents into a regulated or long-lived enterprise codebase.

Status of numbers in this document: Figures attributed to the source paper are marked as *reported*. The paper itself describes the empirical base as nascent. Any number not attributed to a source is marked **[illustrative]** and exists only to show the shape of a target or a worked calculation — do not quote it as evidence.

How to use this cookbook

This document has six parts and six appendices. They are meant to be read in different ways.

Part	Contents	Read it when
I	Foundations — why SDD, the rigor spectrum, the core loop	Once, before you commit to anything
II	The artifact model — what you actually write, and the traceability spine	Before your first pilot spec
III	SDD mapped onto the SDLC, phase by phase	When designing the delivery process
IV	SDD mapped onto the STLC, phase by phase	When designing the test process
V	Tooling and reference pipelines	During tool selection
VI	The rollout plan, metrics, governance, pitfalls	When you need a 90-day plan and a business case
A-F	Templates, prompts, CI wiring, checklists, RTM, glossary	Continuously, as working material

Every operational section in Parts III and IV follows a fixed **recipe** format so you can lift it straight into a process document:

Recipe Goal — what this step produces **Inputs** — artifacts required before you start **Steps** — the ordered procedure **Output artifact** — what exists when you are done **Exit criteria** — the objective test for "this step is complete" **Failure mode** — the specific way this step goes wrong, and the countermeasure

PART I — FOUNDATIONS

1. The problem SDD solves

1.1 The code-centric default

In conventional delivery, code becomes the de facto truth of a system. Requirements documents drift, design diagrams rot, and tests are frequently written after the implementation they describe. The source paper frames the consequences sharply: when a new joiner asks what a function is supposed to do, the answer is "read the code"; when a stakeholder asks whether the system meets its requirements, somebody has to reverse-engineer intent out of implementation.

Spec-driven development inverts that relationship. The specification becomes the authoritative description of intended behaviour, and code becomes a derived artifact — generated, implemented, or verified *against* the spec. In the paper's formulation, code is the implementation detail of the specification, not the other way around: the spec declares intent and the code realises it.

1.2 Why AI coding agents forced the issue

Spec-first thinking is not new. TDD and BDD have argued for it for two decades. What changed is that large language

models are strong at pattern completion and weak at mind reading.

The paper's worked example is worth internalising because it is the entire business case in miniature. A developer prompts an agent with "add photo sharing to my app." The agent must silently decide: which formats, which permission model, what size ceiling, cloud or local storage, whether to compress. The output is plausible-looking code resting on dozens of unstated and often wrong assumptions. That failure mode is what practitioners call **vibe coding** — loose prompting producing inconsistent or erroneous output.

Re-run the same request against a specification that fixes formats, a size ceiling, a storage layout, a deletion-authority rule and a resize rule, and the agent now has enough constraint to produce code that matches intent. The paper cites controlled studies suggesting human-refined specs can cut errors in LLM-generated code by up to 50% (*reported; the paper explicitly calls this evidence base nascent*).

1.3 The reframing that matters for enterprise delivery

The bottleneck moves. When an agent can emit code faster than a human can type it, throughput is no longer limited by coding capacity. It is limited by **specification quality**. Every hour your organisation spends arguing about what "done" means is now the critical path.

For a Quality Engineering function this is the important strategic point: under SDD, QE stops being a downstream verification gate and becomes an upstream owner of specification testability. That shift is the spine of Part IV.

2. The specification spectrum

Not all spec-driven approaches carry the same cost. The paper defines three levels of rigor, sitting to the right of an ad-hoc baseline. Pick a level deliberately, per system — not per organisation.

2.1 The four positions

Ad-hoc / code-first. Code first, documentation later or never, drift normal. This is the baseline, not a level of SDD.

Spec-first. A specification is written before coding to guide the initial build. After the code exists the spec may or may not be maintained; the value is the initial clarity. Lower maintenance burden than the stronger disciplines. Works particularly well for first-cut feature development with an AI assistant, because the upfront spec stops the agent guessing. It does **not** protect against drift over time.

Spec-anchored. The spec is maintained alongside the code for the system's whole life. A behaviour change requires changing both. Automated checks — usually tests derived from the spec — enforce alignment, so divergence shows up as a failing build rather than as silent rot. The paper names this the sweet spot for most production systems. Cucumber-style BDD and OpenAPI-plus-contract-testing are the canonical implementations.

Spec-as-source. The spec is the only artifact humans edit. Code is entirely generated and never hand-modified; manual editing is prohibited or confined to defined extension points. Drift is eliminated by construction. This draws on Design by Contract thinking and is already routine where generation is mature: server stubs from OpenAPI, certified C from Simulink models. It demands high trust in the generator.

2.2 Comparison table

Dimension	Ad-hoc	Spec-first	Spec-anchored	Spec-as-source
Humans edit	Code	Spec, then code	Spec and code together	Spec only
Source of truth	Code	Code after build	Spec, enforced	Spec, by construction
Drift risk	High	Medium-high	Low (test-enforced)	None by design
Enforcement mechanism	None	Human review	Failing tests in CI	Regeneration
Maintenance cost	Lowest	Low	Moderate, continuous	Moderate, front-loaded in tooling
Documentation trustworthy?	No	Only at t=0	Yes	Yes
Prerequisite	None	Template discipline	CI maturity + test automation	Certified/trusted generator
Typical vehicle	—	User story + AC, Spec Kit /specify	Gherkin + Cucumber; OpenAPI + Specmatic/Pact	OpenAPI codegen; Simulink; TESSl
Fits	Prototypes, one-offs, AI first drafts	New features, greenfield starts	Long-lived production systems	Safety-critical, high-volume interface code

2.3 The selection decision tree

Reproducing the paper's decision logic as an executable procedure:

```
START: new feature or new system
|
├─ Q1. Is AI assistance in play, OR are requirements complex/contested?
|   └─ NO → Ad-hoc is acceptable. Stop. Do not add ceremony.
|   └─ YES ↓
|
├─ Q2. Is the system long-lived, OR does it have multiple maintainers?
|   └─ NO → SPEC-FIRST. Write the spec, build, let it go.
|   └─ YES ↓
|
├─ Q3. Is trustworthy code generation viable for this artifact class?
|   └─ NO → SPEC-ANCHORED. Maintain spec + code, enforce with tests.
|   └─ YES → SPEC-AS-SOURCE. Generate only; never hand-edit.
```

The Golden Rule, from the paper: use the minimum level of specification rigor that removes ambiguity in your context. Spec-first for AI-assisted initial development; spec-anchored for long-lived production systems; spec-as-source only where generation tooling is mature and trusted.

2.4 Where SDD is genuinely overkill

The paper is explicit that SDD is not universally applicable, and a cookbook that pretends otherwise will get your rollout rejected. Do not impose it on:

- Throwaway prototypes — the spec investment is discarded with the prototype.
- Solo, short-lived projects with no maintenance horizon.
- Exploratory coding, where premature specification constrains the learning you are trying to do.
- Simple CRUD with obvious requirements, where misinterpretation is unlikely.

Conversely, the paper identifies where value is clearest: AI-assisted work, complex requirements needing stakeholder validation before code, multi-maintainer systems where specs survive team turnover, integration-heavy systems, regulated domains requiring requirement-to-implementation traceability, and legacy modernisation where extracting a spec from existing behaviour enables confident reimplementations.

Practitioner note for regulated utilities and financial clients. The regulated-domain and integration-heavy categories usually both apply at once, which puts most enterprise systems of record directly in spec-anchored territory. Interface layers within them are often good spec-as-source candidates even when the business logic is not.

3. SDD versus the SRS, HLD and LLD you already write

The most common objection you will face internally: "*we already write specifications — this is SRS/HLD/LLD with new vocabulary.*"

The honest answer, per the paper, is that traditional design documents *are* specifications. The difference is not what gets written. It is how the artifact is used and whether it stays aligned with code. Traditional documents are **advisory**: developers read them, then write code that hopefully matches. SDD specs are **enforced**: tests fail when code diverges, and under spec-as-source the code is regenerated rather than hand-edited.

The paper attributes three concrete additions to SDD over classical design documentation:

1. **Executable specifications.** Traditional specs are read by humans. SDD specs run — as BDD scenarios, API contract tests, or model simulations. Mismatch breaks the build.
2. **CI/CD integration.** Spec validation is embedded in continuous integration, so every commit is checked against the spec and drift surfaces immediately rather than at a quarterly review.
3. **AI consumption.** Traditional design docs were written for human readers. SDD specs are structured so coding agents can consume them and generate code and tests, instead of guessing from vague prompts.

Bryan Finster's widely quoted position, cited in the paper, is that SDD is essentially BDD with new branding. The paper concedes the point and argues the branding still earns its keep: it reminds practitioners that specs are meant to be authoritative rather than advisory, and that modern tooling can now enforce what used to depend on human discipline. Treat SDD as an evolution, not a revolution — that framing will also survive contact with a sceptical architecture review board.

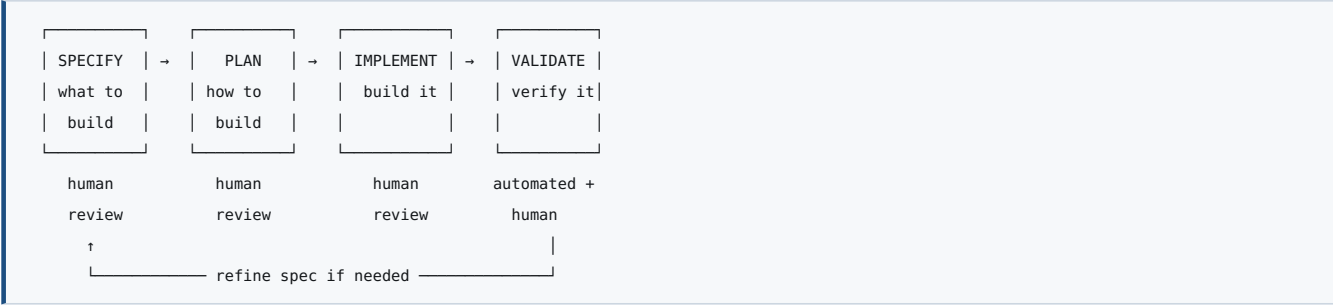
3.1 Mapping your existing artifacts

Do not create a parallel document set. Re-point the ones you have.

Existing artifact	SDD equivalent	What has to change
SRS / BRD	<code>spec.md</code> — behaviour, AC, business rules, edge cases	Strip implementation language; make every requirement individually testable and ID'd
HLD	<code>plan.md</code> — architecture, boundaries, tech choices	Becomes a machine-readable constraint set consumed by agents, not a slide deck
LLD	Mostly absorbed into <code>plan.md</code> + <code>tasks.md</code>	Stop hand-specifying algorithms; specify contracts and invariants instead
Interface spec / IDL	<code>contracts/</code> — OpenAPI, AsyncAPI, protobuf, GraphQL SDL	Must become the generation and contract-test source, not a description of it
Test plan	<code>test-spec.md</code> + Gherkin feature files	Test cases derive from AC IDs rather than being authored independently
Definition of Done	Spec-satisfaction gate in CI	"All scenarios for this AC pass" replaces subjective sign-off
Traceability matrix	RTM generated from IDs, not maintained by hand	Becomes a build output

4. The core loop: Specify → Plan → Implement → Validate

Tooling varies; the loop does not. Each phase emits an artifact that constrains the next, creating a chain of accountability from intent to implementation. Each phase carries a human review checkpoint. Validation is automated **and** human.



4.1 Phase 1 — Specify

Question answered: what should the software do? **Output:** a functional specification of behaviour, requirements and acceptance criteria — *without prescribing implementation*. That separation of "what" from "how" is where SDD's leverage comes from.

Work in this phase: articulate user-facing behaviour as stories and scenarios; define success in Given/When/Then form or as input-output examples; capture business rules and constraints explicitly; and identify edge cases and error conditions upfront rather than discovering them mid-implementation.

The paper's four properties of a good spec — which Appendix D turns into a scored rubric:

- **Behaviour-focused** — describes what happens, not how.
- **Testable** — every requirement is individually verifiable.
- **Unambiguous** — different readers reach the same interpretation.
- **Complete enough** — covers essential cases without over-specifying.

Effective specs also emphasise clarity, modularity and self-checks that help guide agents during implementation.

Calibration rule. Write to the level of detail that removes ambiguity, and stop. If an agent or a developer could read a requirement two ways, clarify it. If only one reasonable reading exists, adding more detail only constrains implementation for no gain.

4.2 Phase 2 — Plan

Question answered: how should we build it? **Output:** a technical plan covering architecture, data models, interfaces and technology choices. Where the spec declares intent, the plan declares *constraints on implementation*.

Work in this phase: select technologies and frameworks; define component architecture and boundaries; design data models and schemas; specify interfaces including APIs, messages and contracts; and identify non-functional requirements for performance, security and scalability.

The plan is what stops a technically correct implementation from violating your standards. It encodes rules of the form "PostgreSQL for persistence" or "every endpoint requires authentication." For an agent, the plan supplies the structural context a functional spec cannot: without it, even a perfect spec can yield code that contradicts your architecture decisions.

4.3 Phase 3 — Implement

Question answered: does working code exist that realises the spec within the plan's constraints?

Procedure: break the plan into discrete, reviewable tasks; implement each — human, agent, or hybrid; review code against **both** spec and plan; and write unit tests that encode spec requirements as executable assertions.

The governing principle is small, validated increments. Each task should deliver a testable slice, so that human checkpoints are frequent and drift is caught before it compounds. The paper's mechanism for why this works with agents: specifications act as **super-prompts** that decompose complex problems into modular components sized to an agent's context window, letting AI handle complexity that would overwhelm a single-shot prompt.

4.4 Phase 4 — Validate

Question answered: does the code actually meet the spec?

Scope: automated tests at unit, integration and acceptance levels; BDD scenarios executed against the implementation; review against non-functional requirements; and stakeholder acceptance testing where appropriate.

The decision rule when validation finds a gap is the single most important governance point in SDD, and Part IV builds a whole defect-arbitration process on it:

- If the spec was wrong or incomplete → **update the spec**.
- If the code fails a valid spec → **fix the code**.
- Either way, **the spec remains the authority**.

That discipline is what keeps specs trustworthy over time: violations get detected and resolved rather than quietly ignored.

4.5 How the loop amplifies AI agents

The paper's mechanisms, restated as things you can design for:

Mechanism	What to build for it
Specs as super-prompts	Keep each spec module small enough to fit an agent's working context
Self-verification	Ship a machine-readable AC checklist with each task so the agent can check its own output
Parallel agent execution	Partition work at the spec level into non-overlapping tasks; orchestrate the dependency edges
Self-spec workflows	Let the agent draft a spec from a high-level prompt, then have a human review and refine it before any implementation — this catches requirement misunderstandings before code exists
Non-determinism control	Use property-based testing to assert spec invariants hold regardless of which valid implementation the agent produced
Safety-critical assurance	Combine agent generation with formal verification for standards compliance (the paper cites ISO 26262)
Brownfield leverage	Encode legacy constraints in the spec so the agent inherits them instead of violating them

The net effect the paper claims: agents move from reactive tools to proactive collaborators. The net effect you should plan for: your agent output quality is now a function of your spec quality, which makes spec review a first-class engineering activity with the same rigor as code review.

PART II — THE ARTIFACT MODEL

5. The spec stack

SDD fails in practice for a boring reason: teams are unclear on which file a given fact belongs in, so facts get duplicated, then diverge. Fix that first by defining a small, non-overlapping artifact set with one owner each.

5.1 Recommended repository layout

```

repo/
├── .sdd/
│   ├── constitution.md      # org-wide, rarely changes
│   └── rubric.md            # spec quality scoring rubric
├── specs/
│   └── FEAT-114-payment-retry/
│       ├── spec.md          # WHAT – behaviour, AC, rules, edge cases
│       ├── plan.md          # HOW – architecture, tech, NFRs
│       ├── tasks.md         # decomposition into reviewable units
│       ├── test-spec.md     # test strategy + AC→test mapping
│       ├── decisions/ADR-007.md # decisions taken during planning
│       └── contracts/
│           ├── payments.openapi.yaml
│           └── retry-events.asyncapi.yaml
├── features/                # Gherkin, one file per spec module
│   └── payment-retry.feature
├── tests/
│   ├── contract/           # Pact / Specmatic
│   ├── property/           # invariant tests
│   └── acceptance/         # step definitions
└── src/

```

5.2 Artifact responsibility matrix

Artifact	Contains	Never contains	Owner	Approves	Change cadence
constitution.md	Non-negotiable org rules: languages, security baseline, logging, banned patterns, review rules	Feature behaviour	Platform/Architecture	CTO or Arch board	Quarterly
spec.md	Behaviour, AC with IDs, business rules, edge cases, error conditions, out-of-scope	Frameworks, class names, SQL, algorithms	Product Owner	PO + QE lead	Per change request
plan.md	Architecture, boundaries, data model, interface list, tech choices, NFR targets	Restated behaviour	Tech Lead	Architect	Per feature
tasks.md	Ordered, independently reviewable units with AC references and dependency edges	New requirements	Tech Lead	Tech Lead	Per sprint
contracts/	OpenAPI / AsyncAPI / protobuf / GraphQL SDL	Prose requirements	API owner	Consumer teams	Versioned
test-spec.md	Level allocation, AC→test-type mapping, data strategy, exit criteria	Behaviour not in spec.md	QE Lead	QE Lead + PO	Per feature
features/*.feature	Executable Gherkin scenarios traceable to AC IDs	Implementation steps	QE + PO jointly	PO	With behaviour
decisions/ADR-*.md	Decision, options rejected, consequences	Requirements	Whoever decided	Tech Lead	Append-only

Rule of one home. A fact lives in exactly one artifact. If a requirement appears in both `spec.md` and `plan.md`, delete it from `plan.md`. Duplication is the seed of drift.

5.3 spec.md — the canonical shape

```

# FEAT-114 – Payment retry on soft decline

## 1. Intent
One paragraph: the user-visible outcome and why it matters.

## 2. In scope / Out of scope
Explicit non-goals. This section prevents agent scope creep.

## 3. Actors and preconditions

## 4. Behaviour
### REQ-114-01 – Soft declines are retried
The system retries a payment when the processor returns a soft
decline code, up to a bounded number of attempts, with increasing
delay between attempts.

    AC-114-01-a GIVEN a payment that returned decline code 51
                WHEN the retry scheduler next runs
                THEN a second authorisation attempt is made
    AC-114-01-b GIVEN a payment that returned decline code 04 (hard)
                WHEN the retry scheduler next runs
                THEN no further attempt is made
                AND the payment is marked terminally failed

### REQ-114-02 – Retries are idempotent
    AC-114-02-a ...

## 5. Business rules
BR-114-01 Maximum 4 authorisation attempts per payment, inclusive of the first.
BR-114-02 Soft-decline code list is owned by the Payments domain, not hardcoded.

## 6. Edge cases and error conditions
EC-114-01 Processor timeout – treated as soft decline.
EC-114-02 Payment cancelled by user mid-retry-window.
EC-114-03 Duplicate webhook for an already-succeeded retry.

## 7. Non-functional expectations (targets only; detail in plan.md)
NFR-114-01 Retry decision latency p99 under 200 ms.

## 8. Open questions
OQ-114-01 Do partial captures count toward the attempt ceiling? – OWNER: PO, DUE: sprint 12

```

Three properties of that template do the heavy lifting:

- **Every statement is addressable.** REQ-, AC-, BR-, EC-, NFR-, OQ- prefixes give you a traceability spine for free.
- **Out-of-scope is explicit.** This is the single highest-value section when an agent is implementing, because it is the only defence against helpful-but-unrequested additions.
- **Open questions are first-class and owned.** Unresolved ambiguity is visible rather than silently resolved by whoever writes the code — human or model.

5.4 The constitution

The constitution is what keeps agent output inside your engineering standards without repeating those standards in every spec. It is short, absolute, and loaded into agent context on every task.

```
# Engineering Constitution – v4
```

```
## Non-negotiable
```

1. Every public endpoint requires authentication and authorisation. No exceptions without an ADR.
2. No secrets in source. Config via the platform secret store only.
3. All persistence is PostgreSQL unless an ADR records an approved exception.
4. Structured JSON logging with a correlation ID on every request path.
5. No new dependency without a licence and CVE check recorded in the PR.
6. Generated code is reviewed by a named human before merge. Agent authorship does not reduce review depth.

```
## Banned patterns
```

- Catch-and-swallow exception handling.
- Business logic inside controllers or database triggers.
- Time-based sleeps in tests.

```
## Conventions
```

- Error responses follow RFC 9457 problem+json.
- Public API versioning in the path (/v1, /v2). Breaking change = new major.

6. Recipe: authoring a specification

Recipe 6.1 — Draft, harden and approve a feature spec

Goal — a `spec.md` that a developer *and* an agent both interpret one way.

Inputs — business intent or epic; `constitution.md`; the relevant domain glossary.

Steps

1. **Capture intent as prose.** One paragraph, stakeholder language. Do not structure yet.
2. **Machine first draft.** Have an agent expand the paragraph into the `spec.md` template (prompt B.1, Appendix B). This is the paper's self-spec pattern: the model drafts the spec, a human reviews before any code exists.
3. **Strip the "how".** Delete every framework name, table name, class name and algorithm. If removing it loses a genuine constraint, that constraint belongs in `plan.md`.
4. **Assign IDs** to every requirement, AC, business rule and edge case.
5. **Adversarial ambiguity pass.** Run prompt B.2: ask a model to find every statement admitting two readings, and every AC not mechanically verifiable. Do not ask it to fix them — ask it to list them.
6. **Resolve or register.** Each ambiguity is either clarified in place or logged as `00-` with an owner and a due date. Never leave it implied.
7. **QE testability review.** The QE lead scores the spec against the rubric in Appendix D. Below threshold, it goes back.
8. **Joint approval.** PO owns correctness of intent; QE lead owns testability. Both sign.

Output artifact — approved `specs/FEAT-xxx/spec.md` at version 1.0.

Exit criteria

- Zero `00-` items without an owner and a date.
- Every `REQ-` has at least one `AC-`.
- Every `AC-` is expressible as a pass/fail check with no human judgement call.
- Rubric score $\geq 80\%$ with no zero-scored dimension.

Failure mode — over-specification. The paper names this first among pitfalls: specs so detailed they become pseudo-code, which defeats the separation of what from how, over-constrains implementation and destroys the abstraction benefit. **Countermeasure:** a hard heuristic — if a reviewer can read your spec and identify the class structure you had in mind, you have written an LLD, not a spec. Cut it.

Failure mode — spec as bureaucracy. Also from the paper: specs degrade into forms to fill in rather than instruments of clarity, at which point teams game or abandon the process. **Countermeasure:** track review cycle time and spec length. If mean spec length grows while defect-escape rate is flat, the added length is ceremony. Delete template sections that never change a decision.

6.1 Worked before/after

Before (ambiguous, untestable):

The system should handle failed payments gracefully and retry them a reasonable number of times before giving up. Performance should be good.

Four unresolvable questions: which failures, what is graceful, how many retries, what is good. An agent will invent all four answers.

After:

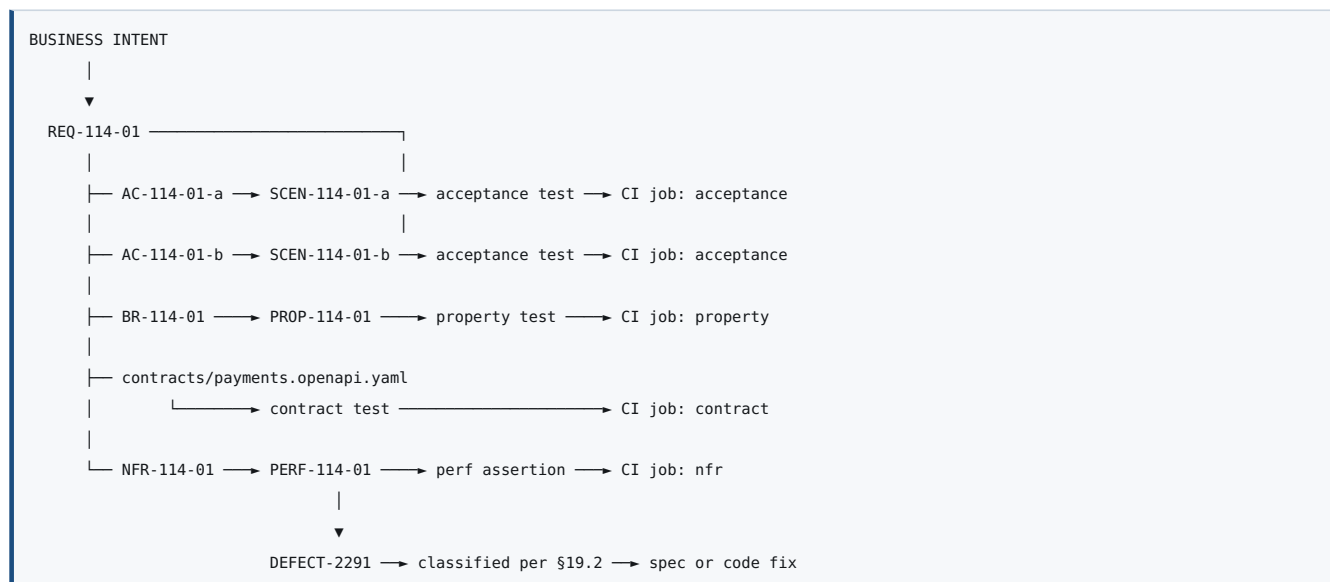
REQ-114-01 Payments returning a soft-decline code are re-attempted up to the ceiling in BR-114-01, with delays per BR-114-03. BR-114-01 Maximum 4 authorisation attempts inclusive of the first. BR-114-03 Delays between attempts: 15 min, 2 h, 24 h. EC-114-01 Processor timeout is classified as a soft decline. AC-114-01-b Hard declines produce no retry and a terminal-failure state. NFR-114-01 Retry decision latency p99 under 200 ms.

Each line is now a test case. That is the actual test of whether a spec is finished.

7. The traceability spine

Traceability is where SDD pays for itself in a regulated environment, and it is the artifact auditors ask for. Under SDD it is a **build output**, not a spreadsheet somebody maintains.

7.1 The chain



7.2 Making it mechanical

Three conventions turn this from aspiration into automation:

1. **ID in the test name or tag.** @AC-114-01-a as a Gherkin tag; test_ac_114_01_a as a function name; a covers: annotation in the test metadata.
2. **A single extractor.** One CI step greps IDs out of spec.md and out of test annotations, joins them, and emits the RTM as JSON plus HTML. Template in Appendix E.
3. **A coverage gate.** The build fails when any AC- in an approved spec has zero linked tests. This is the mechanism that makes "spec coverage" a real number rather than a claim.

7.3 Spec coverage versus code coverage

They answer different questions and you need both.

Code coverage		Spec coverage
Question	Which lines did tests execute?	Which acceptance criteria have a verifying test?
Blind spot	Unwritten code, and code with no requirement behind it	Behaviour nobody specified
Gameable by	Assertion-free tests	Trivially-passing scenarios
Under SDD	Secondary signal	Primary release gate
Target	Team-set [illustrative: 80% lines]	100% of approved AC , no exceptions

A useful third number is **orphan code**: modules with no `REQ-` traceable to them. In brownfield work, orphan code is where undocumented behaviour hides, and it is exactly what §13 addresses.

PART III — SDD ACROSS THE SDLC

Part III maps the four-phase loop onto conventional SDLC stages. Each section is a recipe you can lift into a process document. The RACI convention used: **R**esponsible, **A**ccountable, **C**onsulted, **I**nformed.

8. Stage 1 — Requirements

Recipe 8.1 — Convert intake into an approved spec

Goal — an approved, testable `spec.md` before any design work begins.

Inputs — epic or change request; stakeholder availability; domain glossary; `constitution.md`.

Steps

1. Classify the work against the §2.3 decision tree and record the chosen rigor level in the spec header. Do this *first* — it determines how much of the rest of this cookbook applies.
2. Run Recipe 6.1 to draft and harden the spec.
3. Hold a three-role spec review: PO (is this the intent?), Tech Lead (is this implementable without hidden design?), QE Lead (is every AC verifiable?).
4. Publish the spec to the branch and freeze version 1.0. Changes after this point go through change control, which is the point.

Output artifact — `spec.md` v1.0, approved; ambiguity register with owners.

Exit criteria — Recipe 6.1 exit criteria met; rigor level recorded; RTM skeleton generated with all AC IDs present and zero tests linked (this is the expected initial state).

Failure mode — the spec review becomes a formality where nobody reads the document. **Countermeasure**: require each reviewer to submit at least one written comment or an explicit "no findings" statement with their name on it. Silent approval is not approval.

RACI: PO **A/R** · QE Lead **R** (testability) · Tech Lead **C** · Architect **C** · Delivery Manager **I**

Common questions this stage settles for you: which processor decline codes matter; what the attempt ceiling is; whether partial captures count. Every one of those settled here is a defect you do not pay for in UAT.

9. Stage 2 — Design and planning

Recipe 9.1 — Produce the technical plan and the contracts

Goal — a `plan.md` and a versioned contract set that together constrain implementation without restating behaviour.

Inputs — approved `spec.md`; `constitution.md`; existing architecture and platform standards.

Steps

1. **Derive the interface surface first.** For anything with an API or event boundary, author `contracts/` before prose. Once the contract is agreed, consumer and provider teams can work in parallel — the contract is the agreement, so any implementation matching it is valid by definition.
2. **Circulate contracts to consumers for review before coding.** This is the specific move that produced the paper's reported 75% integration cycle-time reduction in its financial-services case study: incompatibilities surfaced at spec review instead of during integration testing.
3. Write `plan.md`: component boundaries, data model, technology selections, and NFR targets with numbers.
4. Record every contested decision as an ADR. Agents will follow the plan; humans will ask why, six months from now.
5. Validate the plan against the constitution. Any deviation needs an ADR, not a silent exception.
6. Generate mocks and stubs from the contracts immediately. Mock servers generated from specs let dependent work start now (Specmatic and equivalent tools do this directly).

Output artifact — `plan.md`; versioned `contracts/`; ADRs; generated mocks published to the shared environment.

Exit criteria

- Every interface in `spec.md` has a contract entry.
- Contracts reviewed and accepted by each named consumer team.
- Every NFR in `spec.md` has a numeric target and a named measurement method in `plan.md`.
- Zero constitution deviations without an ADR.

Failure mode — the plan re-states behaviour, so behaviour now lives in two files and diverges. **Countermeasure:** a review check that `plan.md` contains no `GIVEN/WHEN/THEN` and no new business rules. If it does, they move to `spec.md` or get deleted.

RACI: Tech Lead **A/R** · Architect **R** (approval) · API consumers **C** · QE Lead **C** · PO **I**

10. Stage 3 — Task decomposition

Recipe 10.1 — Decompose the plan into agent-sized tasks

Goal — an ordered task list where each item is independently implementable, independently reviewable, and traceable to AC IDs.

Inputs — `spec.md`; `plan.md`; `contracts/`.

Steps

1. Break the plan into discrete units. Sizing rule: one unit fits comfortably in an agent's working context together with the constitution and the relevant spec slice.
2. Annotate each task with the `AC-` and `BR-` IDs it satisfies. A task satisfying nothing is either infrastructure or scope creep — label it accordingly.
3. Build the dependency graph and mark the **non-overlapping** sets. Independent sets can be handed to parallel agents; the paper identifies spec-level partitioning as the enabler for parallel agent execution with orchestration only on the dependency edges.
4. Attach a per-task machine-readable acceptance checklist so the agent can self-verify before handing back.
5. Cap work-in-progress. Small validated increments with frequent human checkpoints are what stop drift compounding.

Output artifact — `tasks.md` with IDs, AC links, dependency edges, parallel-safe groupings.

Exit criteria — every AC in scope is covered by at least one task; no task depends on an unimplemented task in the same parallel group; every task carries a self-check checklist.

Failure mode — tasks too large, so the agent runs out of context mid-task, invents the rest, and the review surface becomes too big to inspect properly. **Countermeasure:** an explicit size ceiling (**[illustrative]** e.g. ≤ 400 changed lines, ≤ 6 files). Split on breach, no discussion.

RACI: Tech Lead **A/R** · Developers **C** · QE Lead **C** · Scrum Master **I**

10.1 A task entry that works

```
### T-114-03 – Retry scheduler decision function

Satisfies: AC-114-01-a, AC-114-01-b, BR-114-01, BR-114-03, EC-114-01
Depends on: T-114-01 (decline-code taxonomy)
Parallel-safe with: T-114-05, T-114-06
Constraints: plan.md §4.2 (no scheduler state in the request path)

Self-check before handoff:
[ ] Code 51 → retry scheduled
[ ] Code 04 → no retry, terminal state set
[ ] 4th attempt → no further retry
[ ] Processor timeout → treated as soft decline
[ ] Delay sequence is 15m / 2h / 24h, read from config not literals
[ ] No sleeps in tests
```

11. Stage 4 — Implementation

Recipe 11.1 — Implement a task under SDD

Goal — merged code that satisfies its linked AC without violating the plan or the constitution.

Inputs — one task; `spec.md` slice; `plan.md` constraints; constitution; contracts.

Steps

1. **Assemble the context bundle.** Constitution + relevant spec slice + relevant plan section + the task with its checklist. Not the entire spec — precision beats volume.
2. Implement — human, agent, or hybrid.
3. Write unit tests that encode the spec requirements as executable assertions. Tag them with AC IDs.
4. Agent runs its own checklist and reports each item.
5. **Dual-axis human review:** does this satisfy the spec, *and* does it respect the plan? Both, explicitly. A change can pass tests and still be architecturally wrong.
6. Contract check before merge if the task touches an interface.
7. Merge only with CI green and the RTM showing the task's AC IDs now linked.

Output artifact — merged code; AC-tagged unit tests; updated RTM rows.

Exit criteria — every linked AC has a passing tagged test; contract tests pass; no constitution violation; reviewer has recorded a spec-satisfaction judgement, not just an approval click.

Failure mode — review erosion. Agent-written code is fluent, which makes reviewers skim. This is the most likely way your programme quietly fails. **Countermeasure:** reviewers review against the AC list, not against the diff. Ask "which AC does this line serve?" The constitution rule that agent authorship does not reduce review depth exists for this reason.

Failure mode — silent scope addition. Agents add unrequested features that look like improvements.

Countermeasure: the out-of-scope section in `spec.md`, plus a review question — "is anything here not traceable to an AC?"

RACI: Developer **R** · Tech Lead **A** · Agent operator **R** · Reviewer **R** · QE **C**

12. Stage 5 — Integration, CI/CD and release

Recipe 12.1 — Wire spec enforcement into the pipeline

Goal — drift becomes a build failure instead of a discovery.

Inputs — repository with specs, contracts, features, tests; CI platform.

Steps — build the gate ladder in this order, adding one rung at a time.

Rung	Gate	Fails when	Blocking?
1	Spec lint	Missing IDs, AC with no test tag anywhere, unowned open questions	Yes
2	Contract validation	contracts/ is invalid or has an unapproved breaking change	Yes
3	Unit + property tests	Spec assertions or invariants fail	Yes
4	Contract tests	Implementation deviates from its contract	Yes
5	Acceptance (Gherkin)	Any scenario for an in-scope AC fails	Yes
6	Spec coverage gate	Any approved AC has zero linked passing tests	Yes
7	Drift check (spec-as-source only)	Regenerating from spec produces a diff against committed code	Yes
8	NFR assertions	A measured NFR breaches its target	Yes for release, warn on PR
9	RTM publication	—	No, artifact only

Output artifact — pipeline definition (Appendix C); published RTM per build; drift report.

Exit criteria — all nine rungs exist; rungs 1-7 block merge; a deliberately introduced spec/code divergence is demonstrated to fail the build. **Test your gates by breaking them on purpose** — an unverified gate is a decoration.

Failure mode — gates added faster than the team can keep green, so somebody adds a global bypass.

Countermeasure: one rung per sprint, each with an agreed grace period, and route bypass authority to a named approver who has to record a reason.

RACI: Platform/DevOps **R** · QE Lead **A** · Tech Lead **C** · Release Manager **I**

This rung ladder is the concrete form of the paper's claim that CI/CD integration is one of SDD's three genuine additions over classical design documents: every commit is checked against the spec, so drift surfaces immediately rather than at a periodic review.

13. Stage 6 — Maintenance, change and brownfield adoption

Most enterprise SDD work is not greenfield. The paper treats brownfield as a first-class use case: encode existing behaviour as specifications *before* changing anything, so modernisation preserves required functionality while letting you deliberately drop undocumented behaviour. The spec becomes the bridge between old and new implementations.

Recipe 13.1 — Extract a spec from legacy code

Goal — a `spec.md` describing what the system does today, with intentional behaviour separated from accidental behaviour.

Inputs — legacy source; production logs or traffic samples; existing tests; the humans who remember why.

Steps

1. **Bound the slice.** One module or one interface. Never a whole system.
2. **Harvest observable behaviour** from logs and traffic, not only from code. Traffic tells you what callers actually rely on.
3. **Agent-assisted extraction.** Use prompt B.6 to produce a candidate behaviour catalogue from the code.
4. **Classify every behaviour** into: *intentional* (goes in the spec), *accidental* (a bug callers now depend on — goes in the spec with a `LEGACY-` tag and a deprecation plan), or *dead* (nobody uses it — goes on the removal list with evidence).
5. **Pin the spec with characterisation tests against the current system.** These are your regression net. Run them before you change a line.
6. **Human ratification.** Agent-extracted specs are hypotheses. A domain owner signs off, and disputed behaviour becomes an `0Q-`.
7. Only now make the change, through Recipes 8.1 onward.

Output artifact — legacy `spec.md`; characterisation test suite; classified behaviour register; removal candidate list.

Exit criteria — characterisation suite passes green against the *unmodified* legacy system; every behaviour is classified; the removal list is approved by a business owner.

Failure mode — extracting a spec from code produces a spec that describes the bugs and then blesses them permanently. **Countermeasure:** the three-way classification is mandatory, and every `LEGACY-` tag requires a deprecation owner and target date.

Recipe 13.2 — Change control for an approved spec

Goal — behaviour changes without spec rot. The paper names **specification rot** as the pitfall that specifically kills spec-anchored adoption: teams stop updating specs as code changes, the spec drifts from reality and loses the trust that made it valuable.

Steps

1. Change request arrives → amend `spec.md`, increment its version, log the change in a changelog table at the foot of the file.
2. Impact analysis from the RTM: which AC, tests, contracts and tasks are affected. This is a query, not a workshop, because the IDs exist.
3. Update contracts if the interface moves; run consumer contract tests before merge.
4. Implement via Recipe 11.1.
5. Retire obsolete scenarios explicitly — delete them, and record the deletion. Scenarios for deleted behaviour that still pass are a slow poison.

Exit criteria — spec version incremented; changelog entry present; all impacted RTM rows re-linked; no orphaned scenarios.

Failure mode — code changes first "just this once", spec updated later, never. **Countermeasure:** the automated enforcement the paper prescribes — make drift visible and painful rather than silent and accumulating. Concretely: the spec coverage gate plus a rule that a PR changing behaviour without a corresponding spec diff is auto-labelled `spec-drift` and requires the QE lead's approval to merge.

PART IV — SDD ACROSS THE STLC

This is where SDD changes a Quality Engineering function most, and where the paper's principles need the most translation, because the paper is written for developers. The underlying inversion for QE is this:

Under code-centric delivery, QE verifies code against an interpretation of requirements. Under SDD, QE **owns the verifiability of the specification** and the machinery that proves code conforms to it.

Two consequences follow immediately. First, QE moves upstream — spec testability review becomes a gate QE controls. Second, a large share of test *design* becomes derivation from spec artifacts, which shifts QE effort from authoring test

cases to designing the derivation, judging coverage adequacy, and hunting what the spec failed to say.

The STLC mapping at a glance

STLC phase	Classical activity	Under SDD	QE's new leverage point
Requirement analysis	Read requirements, raise queries, build RTM	Score spec testability; gate approval; register ambiguities	Veto on unverifiable AC
Test planning	Write test strategy and estimates	Allocate every AC to a test level; define the gate ladder	Level allocation decisions
Test case design	Author cases manually from requirements	Derive scenarios, contract tests, property tests from spec artifacts	Coverage adequacy judgement
Environment setup	Provision, stub by hand	Generate mocks/stubs from contracts; derive data from schemas	Parallelism unlocked by contracts
Test execution	Execute, log defects	Run the gate ladder; manage agent non-determinism; exploratory testing against spec blind spots	Finding what the spec missed
Test closure	Report coverage, sign off	Publish spec coverage + drift report; arbitrate spec vs code defects	Release authority on spec conformance

14. STLC Phase 1 — Requirement analysis becomes spec testability review

Recipe 14.1 — Score and gate a specification for testability

Goal — no spec reaches implementation with an acceptance criterion that cannot be mechanically verified.

Inputs — draft `spec.md`; the rubric in Appendix D; domain glossary.

Steps

1. **Per-AC verifiability test.** For each AC, answer three questions in writing: what is the observable? where is it observed? what exact value or state constitutes a pass? Any AC that cannot answer all three is rejected.
2. **Ban the weasel vocabulary.** Reject *appropriately, reasonably, gracefully, efficiently, user-friendly, as expected, properly, quickly, robustly, if needed*. Each is an unresolved decision disguised as a requirement, and an agent will resolve it for you, silently.
3. **Oracle check.** For each AC, name the oracle: a fixed expected value, a contract schema, an invariant, a golden file, a reference implementation, or a stated human judgement. An AC with no nameable oracle is not testable.
4. **Negative and edge sweep.** For every happy-path AC, ask for the corresponding failure AC. The spec phase is where edge cases and error conditions are supposed to be identified, rather than discovered during implementation — so QE's job here is to force that to actually happen.
5. **NFR quantification.** Every NFR needs a number, a percentile, a load condition and a measurement point. "Fast" is not an NFR.
6. **Score and decide.** Rubric score $\geq 80\%$ with no zero-scored dimension $\rightarrow$ approve. Otherwise return with the specific failing IDs listed.

Output artifact — testability score sheet; ambiguity register; approved or rejected spec with findings by ID.

Exit criteria — 100% of AC have a named oracle; zero banned terms; every NFR quantified; register entries all owned and dated.

Failure mode — QE is invited to the spec review but has no authority, so findings become "noted" and the spec ships anyway. **Countermeasure:** make QE approval a merge requirement on the spec file itself via branch protection. Authority has to be mechanical to survive schedule pressure.

14.1 The ambiguity register

One table, kept at the foot of `spec.md`, replaces a great deal of email.

ID	Statement	Readings identified	Resolution	Owner	Due	Status
OQ-114-01	"attempts" in BR-114-01	Authorisations only / includes captures	Authorisations only	PO	S12	Closed
OQ-114-02	Retry window vs user cancellation	Cancel wins / in-flight completes	—	PO	S13	Open

An open row blocks implementation of the tasks that touch it, and only those tasks. That precision is what makes the register tolerable to a delivery manager.

15. STLC Phase 2 — Test planning as level allocation

Under SDD, test strategy is largely an allocation problem: each spec statement gets assigned to the cheapest level that can actually verify it, and to a gate.

Recipe 15.1 — Build the test plan from the spec

Goal — a `test-spec.md` in which every spec statement maps to a level, a technique, a tool and a pipeline gate.

Inputs — approved `spec.md`; `plan.md`; `contracts/`; tool inventory.

Steps

1. Enumerate every `AC-`, `BR-`, `EC-` and `NFR-` into a row.
2. Allocate a level using the table in §15.1 below.
3. Choose the technique and name the tool per row.
4. Assign each row to a CI gate rung from §12.
5. **Reserve explicit exploratory capacity.** Justify it with the paper's own warning: a passing spec test only proves the software matches the spec. If the spec is wrong, the code will faithfully implement the wrong thing. Exploratory testing is your only systematic defence against that, and it is the line item that gets cut first unless it is named in the plan.
6. Define exit criteria in spec terms — see §15.2.
7. Estimate. Note that derivation shifts effort from authoring to reviewing, which changes the shape of your estimate, not necessarily the total.

Output artifact — `test-spec.md` with a complete allocation table, exploratory charters, exit criteria.

Exit criteria — zero unallocated spec statements; every row has a named tool and gate; exploratory charters written against the spec's weakest areas.

Failure mode — over-allocating to the acceptance level, producing a slow, brittle Gherkin suite that nobody trusts and everybody reruns. **Countermeasure:** the allocation table's default is the lowest viable level. Escalate only with a stated reason in the row.

15.1 Allocation table

Spec artifact	Default level	Technique	Typical tool	Gate rung
<code>AC-</code> with pure input→output	Unit	Example-based assertions	pytest / JUnit	3
<code>AC-</code> describing user-observable behaviour	Acceptance	Gherkin scenario	Cucumber / Reqroll / Behave	5
<code>BR-</code> stating an invariant or bound	Property	Property-based testing	Hypothesis / jqwik / fast-check	3
<code>contracts/</code> endpoint or message	Contract	Contract verification	Pact / Specmatic	4
Consumer expectation of a provider	Contract (consumer-driven)	Pact broker flow	Pact	4
<code>EC-</code> error or edge condition	Unit, then acceptance if user-visible	Negative testing, boundary analysis	pytest / Cucumber	3, 5
<code>NFR-</code> latency / throughput	Performance	Load profile + assertion	k6 / JMeter / Gatling	8
<code>NFR-</code> security	Security	SAST/DAST + targeted abuse cases	platform standard	8
Cross-feature state interactions	Integration	Scenario chains	platform standard	5
Spec blind spots (unspecified behaviour)	Exploratory	Charter-based session testing	human, time-boxed	outside gates
Spec-as-source modules	Drift	Regenerate and diff	generator + CI diff	7

15.2 Exit criteria expressed in spec terms

Replace narrative exit criteria with these. They are all computable, which means they cannot be negotiated at the release meeting.

#	Criterion	Threshold
1	Approved AC with at least one passing linked test	100%
2	Contract tests passing for every contract in scope	100%
3	Open <code>0Q-</code> items affecting in-scope AC	0
4	Property tests for every <code>BR-</code> stating a bound or invariant	100%
5	NFR targets measured and met	100% of in-scope NFRs
6	Spec/code drift findings outstanding	0
7	Exploratory charters executed	100% of planned
8	Defects classified per §19.2 with no UNCLASSIFIED remaining	0 unclassified
9	Severity 1 or 2 defects open	0
10	Orphan scenarios (tests for retired behaviour)	0

16. STLC Phase 3 — Test design as derivation

The mechanical core of SDD for QE: test artifacts are derived from spec artifacts rather than invented alongside them. Derivation is the paper's BDD lineage made operational — Gherkin scenarios are executable specifications that bridge business requirement and technical implementation, and under SDD tooling an agent can also generate code from them.

Recipe 16.1 — Derive acceptance scenarios from acceptance criteria

Goal — one traceable, executable scenario per AC, readable by the PO.

Inputs — approved `spec.md`; `test-spec.md` allocation rows at level "Acceptance".

Steps

1. Draft with an agent using prompt B.3, one AC at a time. One AC, one scenario — resist bundling.
2. Tag every scenario with its AC ID. `@AC-114-01-a`. This tag is the traceability link; without it the RTM cannot be generated.
3. **Human rewrite for domain language.** Generated Gherkin drifts toward implementation vocabulary ("when the retry service is invoked"). Rewrite in the business's words ("when the retry window opens"). This matters because the scenario's value is that a PO can validate it.
4. **PO validation.** The PO confirms the scenario captures their intent. The paper's enterprise case study turns on exactly this: product managers wrote Gherkin, developers automated it, and a feature was done only when its scenarios passed — which gave both sides an objective, shared definition of completion.
5. Implement step definitions. Steps call the application through its real boundary, never through internals.
6. Add the boundary and negative variants the AC implies.

Output artifact — `features/*.feature` with AC tags; step definitions; PO validation record.

Exit criteria — every acceptance-level AC has exactly one tagged primary scenario; all scenarios pass; PO has signed the scenario set; no scenario references an internal class or table name.

Failure mode — Gherkin written as UI scripts ("when I click the blue button"), which couples your specification to a layout. **Countermeasure:** ban UI mechanics from step text. Steps describe intent; page objects handle mechanics.

Worked derivation

Spec:

```
AC-114-01-b  GIVEN a payment that returned decline code 04 (hard)
              WHEN the retry scheduler next runs
              THEN no further attempt is made
              AND the payment is marked terminally failed
```

Derived scenario:

```
@AC-114-01-b @FEAT-114
Scenario: A hard decline is not retried
  Given a payment was declined with a hard decline code
  When the retry window opens
  Then no further authorisation is attempted
  And the payment is in a terminally failed state
```

Derived boundary siblings (from BR-114-01, ceiling of 4):

```
@AC-114-01-a @boundary  third attempt still permitted
@BR-114-01   @boundary  fourth attempt is the last
@BR-114-01   @boundary  fifth attempt never occurs
```

Recipe 16.2 — Derive contract tests from the contract

Goal — CI proves every implementation matches its published contract, so integration surprises stop existing.

Inputs — versioned `contracts/`; provider and consumer services; contract testing tool.

Steps

1. Treat the contract as the sole authority: any implementation matching the contract is valid by definition, which means disagreements go to the contract, not to a meeting.
2. Generate provider verification tests from the spec so the provider build fails on deviation.
3. Generate mock servers from the same spec for consumers, so consumer teams develop against the contract rather than against a running provider.
4. For consumer-driven flows, publish consumer expectations to a broker and verify providers against all registered consumers.
5. Gate breaking changes: a contract change that breaks a registered consumer fails the build. Version bump plus migration plan, or no merge.
6. Run contract verification on **every commit**, not nightly. The mechanism only works if deviation is caught immediately.

Output artifact — provider verification suite; published mocks; broker registrations; breaking-change gate.

Exit criteria — every contract has provider verification in CI; every consumer either has a registered expectation or a documented waiver; a deliberately introduced breaking change is demonstrated to fail the build.

Failure mode — contract tests written by hand from the contract, so they encode a misreading of it and drift from it independently. **Countermeasure**: generate from the spec file; never transcribe it.

Recipe 16.3 — Derive property tests for invariants and agent non-determinism

Goal — assert that spec invariants hold regardless of which valid implementation an agent produced.

Why this specifically matters under SDD. The paper names LLM non-determinism as an open challenge — even structured specs can yield varying outputs — and identifies property-based testing as the technique that addresses it, by automatically verifying that spec invariants are satisfied regardless of implementation variation. For a QE function operating agent-generated code, PBT stops being a niche technique and becomes core kit.

Steps

1. Extract invariants from **BR-** statements. Candidates: bounds ("never more than 4 attempts"), conservation ("sum of splits equals the total"), idempotence ("reprocessing the same webhook changes nothing"), monotonicity, round-trip ("parse then serialise returns the input"), ordering.
2. Write each invariant as a property over generated inputs, not over three hand-picked examples.
3. Model realistic generators from the schemas in **contracts/** — including nulls, boundaries, unicode, and the malformed cases your real traffic contains.
4. Pin every failing case as a permanent regression example with its **BR-** ID.
5. Set shrinking on, and require the minimal counterexample in the defect report.

Output artifact — property suite keyed to **BR-** IDs; pinned counterexample corpus.

Exit criteria — every **BR-** expressing a bound or invariant has a property test; each property runs a stated minimum number of cases in CI (**[illustrative]** 200 on PR, 5,000 nightly).

Failure mode — properties so weak they pass on any implementation ("result is not null"). **Countermeasure:** mutation-test the properties, or manually break the implementation and confirm the property catches it.

Recipe 16.4 — Charter exploratory testing against spec blind spots

Goal — find the behaviour that matters and that nobody specified.

Why it is mandatory, not optional. This directly answers the paper's subtlest pitfall, **false confidence**: a passing spec test guarantees only that software matches the spec; if the spec is wrong, the code faithfully implements the wrong thing. Specifications need the same careful review as code, and they are not a substitute for human judgement about requirements. Exploratory testing is how a QE function operationalises that warning.

Steps

1. Write charters that target the spec's weak points, not its strong ones: sections with the lowest rubric scores, areas with closed-but-contested **OQ-** items, interactions between features that no single spec owns, and anything the spec declares out of scope but users will reach anyway.
2. Time-box sessions (**[illustrative]** 90 minutes) with written notes.
3. Route every finding through the §19.2 classifier. Most exploratory findings under SDD are **spec defects**, not code defects — that is the signal you are looking for.
4. Feed confirmed gaps back as spec amendments via Recipe 13.2, and add the derived test at the level §15.1 prescribes.
5. Track "spec gaps found per session" as a health metric. A falling trend is real maturity; a zero is usually a charter problem.

Output artifact — charters, session notes, classified findings, resulting spec amendments.

Exit criteria — all planned charters executed; every finding classified; spec amendments raised for confirmed gaps.

17. STLC Phase 4 — Environment and test data from the spec

Recipe 17.1 — Stand up environments from contracts and schemas

Goal — dependent teams test in parallel, without waiting for each other's services.

Inputs — `contracts/`; data schemas from `plan.md`; environment platform.

Steps

1. Generate mock servers for every external and internal dependency directly from its contract. This is the specific enabler in the paper's API-first case study: spec-generated mocks let frontend work proceed in parallel with backend work.
2. Version mocks with the contract. A stale mock is worse than no mock, because it produces confident false passes.
3. Derive test data from schemas and from the examples embedded in the spec. Spec examples become fixtures — that way a spec change breaks the fixtures visibly.
4. Generate boundary datasets from schema constraints: min, max, min-1, max+1, null, empty, maximum length, encoding edge cases.
5. Compliance check the data. Utilities and financial clients: no production-derived personal data in lower environments without masking approved by the data owner. Synthetic generation from schemas is the SDD-native answer, and it is a genuine side benefit of having schemas.
6. Make environment setup reproducible from committed config, and treat the config as a spec-adjacent artifact under the same review.

Output artifact — versioned mocks; synthetic datasets; boundary corpus; reproducible environment definitions.

Exit criteria — every dependency has a contract-generated mock at the correct version; zero unmasked production data below production; environment can be rebuilt from committed config with no manual step.

Failure mode — mocks drift from the contracts they came from, so integration failures reappear in a new costume.

Countermeasure: regenerate mocks in CI from the contract on every build. Never hand-edit a generated mock.

18. STLC Phase 5 — Execution

Recipe 18.1 — Execute the gate ladder and manage agent variability

Goal — a deterministic, trustworthy signal from a partly non-deterministic production process.

Inputs — the \$12 gate ladder; derived test suites; environments.

Steps

1. Run gates in cost order: lint → unit/property → contract → acceptance → NFR. Fail fast on the cheapest.
2. **Enforce a strict flaky-test policy.** Under SDD a flaky acceptance test is more corrosive than usual, because the suite is the specification's proof. Quarantine within one day, fix or delete within an agreed window (**[illustrative]** 5 working days). Never re-run to green.
3. **Distinguish non-determinism in the product from non-determinism in generation.** If two agent attempts produce different-but-valid implementations, your property tests should pass on both. If a property passes on one and fails the other, either the invariant was underspecified or one implementation is wrong. Both are findings — record which.
4. **Verify agent self-checks independently.** A self-reported checklist is evidence, not proof. Sample and audit (**[illustrative]** 20% of tasks) to measure how often self-verification is wrong; that number is a real input to how much you can lean on agents.
5. Publish spec coverage per build, not per release.
6. Execute exploratory charters in parallel with the automated ladder. Automated gates confirm the spec; humans find what it omitted.

Output artifact — per-build results, spec coverage number, drift report, classified defects, exploratory notes.

Exit criteria — all blocking rungs green; zero quarantined tests covering an in-scope AC; self-check audit sample complete.

Failure mode — the acceptance suite gets slow, so somebody moves it out of the PR gate and into nightly; drift detection latency goes from minutes to a day and the enforcement value collapses. **Countermeasure:** a suite runtime budget (**[illustrative]** 10 minutes on PR) enforced as a gate of its own. When you breach it, push tests down a level rather than pushing them out of the pipeline.

18.1 Agent-executed testing — what to allow

Agents are useful in test execution, unevenly. Allocate trust deliberately.

Activity	Agent suitability	Required human control
Drafting scenarios from AC	High	Review for domain language and coverage adequacy
Writing step definitions	High	Review for boundary leakage into internals
Generating boundary datasets	High	Review realism against production distributions
Writing property generators	Medium	Verify the generator actually reaches the interesting space
Triaging failures to a probable cause	Medium	Human confirms before a defect is raised
Classifying spec vs code defect	Low	Human decision — this determines whether the spec changes
Deciding release readiness	None	Human, against §15.2 criteria

The one row that must not be delegated is defect classification, because it is the point at which the authority of the specification is exercised.

19. STLC Phase 6 — Closure, defect arbitration and reporting

19.1 Test closure

Recipe 19.1 — Close a cycle

Goal — an evidence pack that supports release sign-off and an audit request with no additional work.

Steps

1. Publish the generated RTM: `REQ → AC → scenario/test → result`.
2. Publish spec coverage, drift findings, and the defect classification breakdown.
3. Evaluate the §15.2 exit criteria as a computed checklist.
4. Retire obsolete scenarios and record retirements.
5. Run a spec-quality retrospective: which spec sections produced the most defects? Those sections' *authors and templates* are the improvement target, not the testers.
6. Update the rubric and templates with what you learned. This is the compounding loop.

Output artifact — release evidence pack: RTM, coverage, drift, defect classification, exploratory notes, exit-criteria evaluation.

Exit criteria — all ten §15.2 criteria evaluated with evidence; evidence pack archived against the release tag.

19.2 Defect arbitration — the highest-value process in Part IV

The paper's validation rule is short: when code does not meet the spec, either the spec was wrong (update it) or the code was wrong (fix it), and either way the spec remains the authority. Turning that into a working triage process is what actually changes behaviour in a delivery organisation, because it forces the question "what did we say?" before "who broke it?"

Classification, applied at triage. No defect leaves triage unclassified.

Class	Definition	Route to	Fix is	Metric it feeds
CODE	Spec is correct and unambiguous; implementation does not satisfy it	Developer / agent operator	Change code	Implementation defect density
SPEC-GAP	Behaviour was never specified; no AC covers it	PO + QE	Amend spec, then derive test, then code	Spec completeness — the key SDD metric
SPEC-WRONG	Spec specified it, and the specified behaviour is itself incorrect	PO	Amend spec, retire the wrong scenario, re-derive	Spec correctness
SPEC-AMBIGUOUS	Spec admits two readings; implementation chose the other one	PO + QE	Clarify spec, then align code	Ambiguity escape rate
PLAN-VIOLATION	Behaviour is correct but the implementation breaches an architectural constraint or the constitution	Tech Lead	Refactor	Constraint adherence
CONTRACT	Implementation deviates from its published contract	API owner	Fix implementation, or version the contract	Contract conformance
GENERATION	Spec is correct and the generator produced wrong code from it (spec-as-source)	Tooling owner	Fix generator or template — never patch generated code	Generator trust
TEST	Product is correct; the test or its oracle is wrong	QE	Fix the test	Test defect rate
ENV/DATA	Failure caused by environment or data, not by the product	Platform	Fix environment	Environment stability

Triage decision sequence — work down; first match wins:

1. Is the test itself correct?	no → TEST
2. Is the environment/data sound?	no → ENV/DATA
3. Does an AC cover this behaviour?	no → SPEC-GAP
4. Does the AC admit more than one reading?	yes → SPEC-AMBIGUOUS
5. Is the specified behaviour itself right?	no → SPEC-WRONG
6. Was code generated from the spec?	yes and code ≠ spec → GENERATION
7. Does it deviate from a published contract?	yes → CONTRACT
8. Does it breach plan/constitution?	yes → PLAN-VIOLATION
9. Otherwise	→ CODE

Why this pays. The ratio of SPEC-classified to CODE-classified defects is the single most informative number an SDD programme produces. A high and *falling* spec-defect share means your spec process is learning. A high and *flat* share means you have automated the production of code that faithfully implements the wrong requirements — which is the paper's false-confidence pitfall showing up in your defect data, where you can actually see it.

19.3 Metrics that matter

Metric	Definition	Direction	Why it matters
Spec coverage	Approved AC with ≥1 passing linked test ÷ all approved AC	→ 100%	Primary release gate
Spec-defect share	(SPEC-GAP + SPEC-WRONG + SPEC-AMBIGUOUS) ÷ all defects	↓ over time	Spec process maturity
Ambiguity escape rate	SPEC-AMBIGUOUS defects ÷ AC shipped	↓	Effectiveness of Recipe 14.1
Drift detection latency	Time from divergence introduced to build failure	↓ to minutes	Whether enforcement is real
First-pass agent yield	Agent tasks accepted at first review ÷ all agent tasks	↑	Spec quality as consumed by agents
Rework ratio	Effort re-doing accepted work ÷ total effort	↓	The cost SDD is meant to remove
Contract conformance	Contracts with passing provider verification ÷ all	100%	Integration risk
Spec-gap discovery rate	Spec gaps found per exploratory session	↓ over time	Whether specs are getting more complete
Orphan code	Modules with no traceable <code>REQ-</code>	↓	Undocumented behaviour exposure
Review depth	Reviewer comments per agent-authored PR vs human-authored	≈ parity	Early warning for review erosion

Baseline all ten before the pilot starts. Without a baseline you will be unable to defend the programme at the first budget review, and "it feels better" is not a defence.

PART V — TOOLING AND REFERENCE PIPELINES

20. The tool landscape

The paper's tool taxonomy, with selection guidance added.

Category	Examples	Role in SDD	Rigor level it enables
BDD frameworks	Cucumber, SpecFlow/Reqnroll, Behave	Executable specs in plain language (Gherkin)	Spec-anchored
TDD frameworks	RSpec, JUnit, pytest	Encode specifications as unit tests	Spec-first → anchored
API specification	OpenAPI/Swagger, GraphQL SDL, Protocol Buffers, AsyncAPI	Define contracts; generate code and tests	Anchored → as-source
Contract testing	Pact, Specmatic	Verify implementations match specs	Spec-anchored
AI-assisted SDD	GitHub Spec Kit, Amazon Kiro, Tessl	Structured AI workflows from spec to code	Spec-first → as-source
Model-based design	Simulink, SCADE	Visual specs that generate embedded code	Spec-as-source
Property-based testing	Hypothesis, jqwik, fast-check	Verify invariants hold across implementations	All levels; essential with agents

20.1 What each family actually gives you

BDD frameworks. Gherkin's `Given/When/Then` scenarios serve two purposes simultaneously: documentation stakeholders can read, and automated tests that verify code. Cucumber (Ruby/Java/JavaScript), Reqnroll — the successor to SpecFlow — for .NET, and Behave for Python execute these against the application. The paper's own practitioner guidance is worth repeating verbatim in your process doc: BDD scenarios are specifications, not just tests. Write them before implementation, involve stakeholders in writing them, and treat them as the authoritative description of behaviour.

API specification tools. Spec-driven API work predates the SDD label under the names design-first and API-first. OpenAPI lets you define endpoints, schemas and examples, then generate server stubs, client SDKs and documentation from the spec. GraphQL SDL makes the schema the frontend/backend contract and unlocks parallel development. AsyncAPI does the same for event-driven architectures. Protocol Buffers and gRPC define service interfaces and message types with strongly-typed client and server code generated automatically.

Contract testing. Pact and Specmatic automate the verification that implementations actually match their specs — which is the difference between having a contract and having an *enforced* contract.

AI-assisted SDD toolkits. These encode the insight that multi-step prompting with explicit artifacts beats single-shot "just code this":

- **GitHub Spec Kit** — an open-source toolkit with four explicit phases: `/specify` generates a detailed spec from a prompt, `/plan` creates technical architecture, `/tasks` breaks the plan into implementation tasks, then implementation generates code task by task, with human review and refinement at each phase.
- **Amazon Kiro** — walks users through requirements, design and task creation before any code generation, with explicit staging that prevents the model guessing at requirements never specified.
- **Tessl** — the most radical position: spec-as-source, where the spec is the maintained artifact and code is regenerated from it; developers edit specs, never generated code.

Model-based design. Simulink and SCADE are the mature, certified end of spec-as-source, and the reason the paper can claim the level is practical rather than speculative.

20.2 A pragmatic starting stack

For an enterprise Java/.NET/Python estate adopting spec-anchored delivery, the smallest stack that closes the loop:

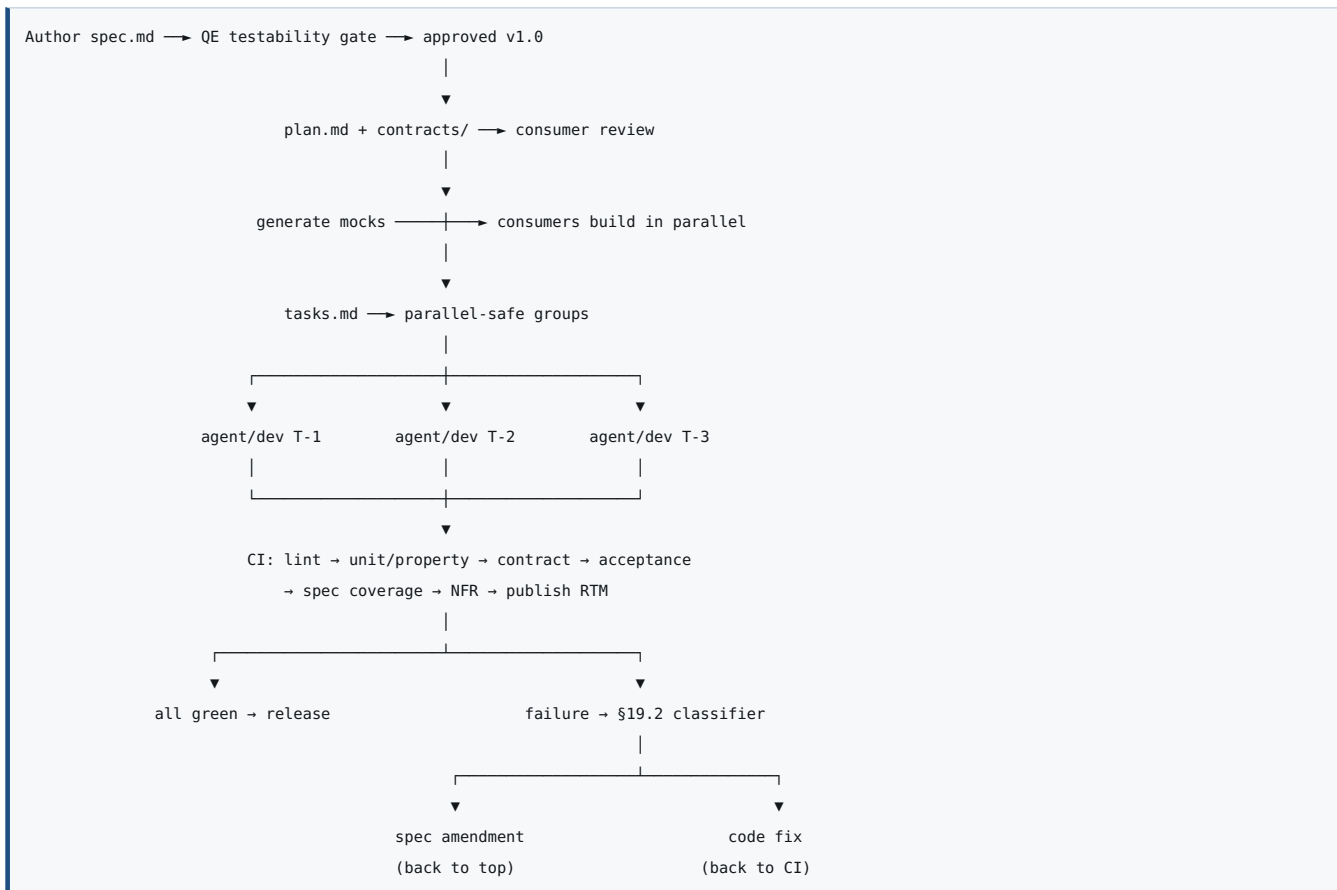
Need	Pick	Rationale
Spec authoring workflow	GitHub Spec Kit conventions, even without the tool	Four phases map 1:1 to the core loop; the conventions are the value
Interface contracts	OpenAPI 3.1 + AsyncAPI for events	Widest generation and tooling support
Contract enforcement	Specmatic or Pact	Specmatic if your contracts are spec-first; Pact if consumer-driven
Acceptance execution	Cucumber / Reqnroll / Behave, matching your stack	PO-readable, AC-taggable
Invariants	Hypothesis / jqwik / fast-check	Non-negotiable once agents generate code
Traceability	Home-built extractor over ID tags (Appendix E)	Avoids buying an ALM module you will outgrow
Agent harness	Your existing coding agent, plus a constitution file	The constitution does more for output quality than tool choice

Deliberately excluded from the starting stack: heavyweight ALM requirement modules, bespoke spec DSLs, and any spec-as-source generator for business logic. Add them only if a specific failure demands them.

Failure mode — tooling complexity. The paper flags this directly: AI-assisted tools generate elaborate artifacts, and teams drown in generated plans, task lists and intermediate documents. Its prescription is to start simple and add complexity only where it demonstrably helps, and to avoid cargo-culting elaborate workflows that add process without value. **Countermeasure:** for every generated artifact type, name the decision it changes. If you cannot, stop generating it.

21. Reference pipelines

21.1 Spec-anchored pipeline (the default target)



21.2 Spec-as-source pipeline (interfaces and generated layers only)



The drift check is the whole mechanism. Without it, someone patches generated code under deadline pressure, and you have silently reverted to code-first while still claiming spec-as-source.

21.3 What the paper's case studies tell you to copy

Case study 1 — API-first microservices, financial services, spec-anchored with OpenAPI. The problem was integration hell: services failed on joint deployment because teams made incompatible contract assumptions, surfacing only at integration testing and requiring expensive rework. The intervention was mandated API-first: OpenAPI specs before implementation, consumer teams reviewing specs and giving feedback before any coding, Specmatic generating mocks so frontend and backend proceeded in parallel, and Specmatic validating in CI that implemented services matched their specs — with any deviation failing the build. Reported outcome: integration failures dropped sharply and cycle time for API changes fell 75%, because incompatibilities were caught at spec review rather than in production (*reported figure*).

Copy this: contract-before-code, consumer review as a gate, generated mocks for parallelism, contract verification in CI. That is the highest return-per-unit-effort move in this entire cookbook for an integration-heavy estate.

Case study 2 — BDD for enterprise features, spec-anchored with Cucumber. The problem was that developers and product managers disagreed on what "done" meant; QA found mismatches against product expectation and arguments followed about whose interpretation was authoritative. The intervention: PMs wrote Gherkin scenarios in plain language, developers implemented step definitions, and a feature was done only when all its scenarios passed. The scenarios became a shared language both business and technical stakeholders could read and validate; when disputes arose the scenario was the authority — wrong scenario, update it with explicit stakeholder agreement; wrong code, fix the code.

Copy this: the arbitration rule. It is the cultural heart of SDD and costs nothing but discipline.

Case study 3 — model-based embedded, spec-as-source with Simulink. The problem was ISO 26262 certification requiring every line of code traced to requirements, which is punishing when code is hand-written. The intervention: control algorithms modelled as block diagrams with state machines, the model *being* the specification; behaviour simulated and verified at model level before any code existed; then certified code generation, with the generation itself certified so the generated C was guaranteed to behave as the model specified. Engineers never edited generated code — logic changes meant changing the model and regenerating, keeping verified model and deployed code aligned by construction.

Copy this: the "verify at the spec level before code exists" move. Even without Simulink, you can simulate against contracts and validate scenarios with the PO before implementation — which is the same idea at lower cost.

PART VI — EXECUTION: ROLLOUT, GOVERNANCE, PITFALLS

22. The 90-day implementation plan

One pilot, one service, four stages. Each stage has entry and exit criteria so the programme can be stopped or extended on evidence rather than sentiment.

Stage 1 — Days 1-15: Assess and baseline

Entry	Executive sponsor named; pilot budget approved
Activities	Inventory candidate systems and place each on the §2 spectrum. Pick one pilot: integration-heavy, actively changing, with a co-operative PO. Baseline all ten §19.3 metrics. Draft <code>constitution.md v1</code> . Draft spec/plan/task/test-spec templates. Audit tooling gaps. Name the QE lead who owns the testability gate.
Exit	Pilot selected with written rationale; ten metrics baselined with evidence; constitution v1 approved by architecture; templates in the repo; rigor level chosen per candidate system
Do not	Do not attempt a multi-team rollout. Do not pick your most critical system. Do not pick your least important one either — nobody learns from a system nobody cares about.

Stage 2 — Days 16-30: Pilot at spec-first

Entry	Stage 1 exit met; pilot team allocated ([illustrative] 4-6 engineers + 1 QE + 1 PO)
Activities	Run Recipes 8.1 → 11.1 on two real features. QE runs the testability gate for real, with authority. Stand up the RTM extractor. Add gate rungs 1-3. Hold a retrospective after each feature, not at the end.
Exit	Two features delivered spec-first; both specs score $\geq 80\%$ on the rubric; rungs 1-3 blocking and demonstrated to fail on a deliberate break; RTM generating per build; retrospective actions logged
Watch for	Spec-as-bureaucracy complaints in the first retrospective. Take them seriously and cut template sections — this is the moment adoption is won or lost.

Stage 3 — Days 31-60: Harden to spec-anchored

Entry	Stage 2 exit met; team not actively hostile
Activities	Author OpenAPI/AsyncAPI contracts for the pilot's interfaces. Get consumer review as a gate. Generate mocks. Add gate rungs 4-6 (contract, acceptance, spec coverage). Introduce property tests for BR- invariants. Introduce §19.2 defect classification at triage. Start exploratory charters. Run change control (Recipe 13.2) through at least one real change request.
Exit	100% spec coverage on pilot features; contract verification in CI for every pilot interface; defect classification running with zero unclassified; at least one spec amendment executed end-to-end through change control; drift detection latency measured in minutes
Watch for	Acceptance suite runtime creeping past the PR budget. Push tests down levels now, before the team pushes them out of the pipeline.

Stage 4 — Days 61-90: Institutionalise and scale

Entry	Stage 3 exit met; metric deltas against baseline available
Activities	Update Definition of Ready ("spec approved, testability gate passed") and Definition of Done ("all AC scenarios passing, RTM complete, zero unclassified defects"). Stand up a lightweight spec review forum. Train two additional teams using the pilot's artifacts as worked examples. Publish the metric delta pack. Assess spec-as-source candidates — interface layers and codegen-friendly boundaries only. Write the brownfield extraction playbook from real experience (Recipe 13.1).
Exit	DoR/DoD updated in the delivery handbook; two further teams onboarded and producing approved specs; metric delta published with baseline comparison; spec-as-source candidate list with a go/no-go per candidate; constitution at v2 reflecting what the pilot learned
Decision point	Continue, adjust, or stop — on the evidence in the metric delta pack. Say this out loud at the start of the programme. A rollout that cannot be stopped will not be believed.

22.1 Effort shape

The shape of the change, not a quantified estimate.

Activity	Under code-first	Under SDD
Requirements	Light, deferred	Heavier, front-loaded
Design	Documented, advisory	Moderate, becomes an enforced constraint set
Implementation	Heaviest	Lighter with agents; review-dominated
Test design	Heavy, manual authoring	Lighter authoring, heavier derivation design and coverage judgement
Test execution	Moderate	Lighter, automated by gate ladder
Defect rework	Heavy, late, expensive	Substantially lighter — the source of the return
Documentation	Written, then rots	Near-zero incremental; the spec <i>is</i> the documentation
Audit and traceability	Expensive, retrospective, manual	Near-zero; RTM is a build output

The business case rests on the last three rows. Anyone who models only the first row will conclude SDD costs more, which is why you baseline rework and audit effort in Stage 1.

23. Roles and the redefinition of work

The paper is explicit that SDD reshapes what developers do: they shift from manual coding toward orchestrating specifications, reviewing AI output and focusing on high-level design. In greenfield work developers become architects who design through specifications rather than code, concentrating on requirements elicitation, constraint definition and acceptance criteria — the what rather than the how — while agents handle translation to implementation, and humans stay responsible for whether specs capture the actual requirements. In brownfield work the new activity is encoding existing behaviour as specs before changing anything.

The paper also names the new challenges honestly: spec maintenance, tool mastery, and the judgement required to know when AI output is correct. Plan for all three.

Role	Loses	Gains	New skill to build
Developer	Volume typing of routine code	Spec authorship input, agent orchestration, deep review	Judging when generated output is correct
Tech Lead	LLD authorship	Constraint design, task decomposition for agents, plan ownership	Context engineering: what to put in an agent's window
Architect	Slide-deck HLDs	Machine-consumable constraint sets, constitution ownership	Writing architecture agents can obey
QE / Test Architect	Manual case authoring from ambiguous requirements	Upstream testability authority; derivation design; defect arbitration	Property-based testing, contract testing, spec rubric design
Test Engineer	Repetitive scripting	Exploratory charters against spec blind spots; oracle design	Exploratory technique; adversarial spec reading
Product Owner	Prose requirements nobody reads	Authorship of executable AC; direct arbitration authority	Writing Given/When/Then that survives automation
BA	Document production	Ambiguity elimination; spec structuring	Formal precision without formal notation
DevOps/Platform	—	Gate ladder, drift detection, generation pipelines	Spec-coverage tooling

The QE career point, stated plainly: SDD moves the QE function's centre of gravity from "find defects in code" to "prevent ambiguity in specs and prove conformance mechanically." That is a promotion in scope. The functions that resist it become script maintainers for tests that agents write faster.

24. Pitfall register

The paper's five pitfalls, plus three you should expect in an enterprise setting, each with a detection signal and a countermeasure. Review this table at every stage gate.

#	Pitfall	What it looks like	Detection signal	Countermeasure
1	Over-specification	Specs become pseudo-code; the what/how separation collapses; implementation is over-constrained and the abstraction benefit is lost	A reviewer can infer your intended class structure from the spec; spec length grows while defect escapes stay flat	Hard rule: no frameworks, class names, SQL or algorithms in <code>spec.md</code> . Cut on sight.
2	Specification rot	Specs not updated as code changes; spec drifts from reality; documentation value and trust erode	Spec coverage falling; PRs changing behaviour with no spec diff	Automated enforcement: make drift a build failure so it is visible and painful rather than silent and accumulating
3	Spec as bureaucracy	Specs become forms to fill in rather than tools for clarity; teams game or abandon the process	Review cycle time rising; identical boilerplate across specs; template sections nobody reads	Minimum needed to remove ambiguity. Delete any template section that has never changed a decision.
4	Tooling complexity	Teams drown in generated plans, task lists and intermediate documents; elaborate workflows cargo-culted	Generated artifacts nobody opens; agent sessions producing more documents than code	Start simple; add tooling only where it demonstrably helps. For each artifact, name the decision it changes.
5	False confidence	Passing spec tests treated as proof of correct software, when they only prove code matches spec; a wrong spec is faithfully implemented	Spec-defect share high and flat; near-zero exploratory findings	Specs get the same review rigor as code. Mandatory exploratory charters (Recipe 16.4). PO validation of scenarios.
6	Review erosion (<i>enterprise addition</i>)	Fluent agent-written code gets skimmed rather than reviewed	Comments per agent-authored PR well below human-authored	Review against the AC list, not the diff. Constitution clause: agent authorship does not reduce review depth. Track review-depth parity.
7	Gate bypass culture (<i>enterprise addition</i>)	Failing gates routinely overridden under deadline pressure	Bypass count per sprint rising; a global skip flag exists	Named bypass approver, recorded reason, bypass count reported at the stage gate
8	Two-speed adoption (<i>enterprise addition</i>)	One team does SDD properly; adjacent teams ignore it; the contract boundary between them becomes the failure point	Integration defects concentrated at the boundary with non-adopting teams	Adopt along interface boundaries, not org charts. Contracts first at every boundary you do not control.

25. Governance

Keep it minimal. Governance that outgrows the engineering it governs is pitfall 3 wearing a lanyard.

Four artifacts, four owners.

Decision	Forum	Cadence	Deciding role
Rigor level per system	Architecture review	On system onboarding	Architect
Constitution changes	Architecture review	Quarterly	Architect / CTO
Spec approval	Async, in the PR on the spec file	Per change	PO (intent) + QE Lead (testability), both required
Spec vs code arbitration on a defect	Triage	Daily	PO decides intent; QE classifies
Gate bypass	Named approver	As needed, counted	QE Lead
Template and rubric changes	Retrospective	Per stage gate	QE Lead

Definition of Ready (spec-anchored):

- Spec approved at $v \geq 1.0$ by PO and QE Lead
- Testability rubric $\geq 80\%$, no zero-scored dimension
- No open `00-` affecting in-scope AC
- Contracts drafted and consumer-reviewed for any interface in scope
- Plan constraints recorded; constitution deviations have ADRs

Definition of Done (spec-anchored):

- Every in-scope AC has a passing, tagged, linked test
- Contract verification green for all touched contracts
- Property tests present for every in-scope `BR-` invariant
- Spec coverage 100% on in-scope AC; RTM published
- Zero unclassified defects; zero open Sev 1/2
- Spec updated and versioned if behaviour changed; obsolete scenarios retired
- NFR targets measured and met

26. Closing: what to hold onto

The paper's conclusion is that SDD inverts the traditional relationship between specification and code — specs become

authoritative, code derivative — and that this matters more as AI coding assistants get more capable, because once an agent generates code faster than a human can type, the bottleneck shifts to specification quality. The three rigor levels give you options from lightweight initial clarity to rigorous generation, mature tooling exists across the spectrum, and teams should match rigor to need rather than over-engineering process. SDD builds on decades of TDD and BDD wisdom; what is new is tooling and AI capability that make specs more powerful than before.

Five things to carry into your own rollout:

- 1. **Pick the level deliberately, per system.** The Golden Rule is the minimum rigor that removes ambiguity. Most enterprise systems land on spec-anchored; interfaces within them are often the only good spec-as-source candidates.
- 2. **Enforcement is the whole difference.** You already write specifications. What SDD adds is that divergence breaks the build. If you implement one thing from this cookbook, implement the gate ladder in §12.
- 3. **QE moves upstream.** The testability gate in Recipe 14.1 and the arbitration process in §19.2 are where a Quality Engineering function converts SDD from a developer practice into an organisational capability.
- 4. **Guard against false confidence.** A green suite proves conformance to the spec, nothing more. Exploratory testing against spec blind spots is not optional overhead; it is the control for the failure mode SDD introduces.
- 5. **Measure the spec-defect share.** It is the one number that tells you whether your specification process is learning or whether you have simply industrialised the construction of the wrong thing.

The question this discipline reframes, in the paper's closing words paraphrased: the developer's central question stops being "what code should I write?" and becomes "what specification should I provide?" Everything in Parts III and IV is machinery for answering the second question repeatably.

APPENDIX A — TEMPLATES

A.1 spec.md

```
# <FEAT-ID> - <Title>

Rigor level: spec-first | spec-anchored | spec-as-source
Spec version: 1.0      Status: Draft | Approved
Owners: PO <name> · QE <name> · Tech Lead <name>

## 1. Intent
## 2. In scope
## 3. Out of scope      <-- agents obey this; keep it strong
## 4. Actors and preconditions
## 5. Behaviour
### REQ-<id>-01 - <statement>
    AC-<id>-01-a GIVEN ... WHEN ... THEN ...
## 6. Business rules      BR-<id>-nn
## 7. Edge cases and errors  EC-<id>-nn
## 8. Non-functional targets  NFR-<id>-nn (value + percentile + load + measurement point)
## 9. Data and states
## 10. Glossary deltas
## 11. Open questions (ambiguity register)
| ID | Statement | Readings | Resolution | Owner | Due | Status |
## 12. Changelog
| Version | Date | Change | Reason | Approved by |
```

A.2 plan.md

```
# <FEAT-ID> – Technical Plan
Spec version this plan targets: <x.y>

## 1. Architecture and component boundaries
## 2. Data model and schemas
## 3. Interfaces (index into contracts/; no prose duplication)
## 4. Technology selections and versions
## 5. NFR approach (how each NFR- target is achieved and measured)
## 6. Security and compliance controls
## 7. Constitution deviations (each requires an ADR reference)
## 8. Risks and mitigations
## 9. Decisions (ADR index)

CHECK BEFORE APPROVAL: this file contains no GIVEN/WHEN/THEN and no new
business rules. Behaviour lives only in spec.md.
```

A.3 tasks.md

```
# <FEAT-ID> – Tasks
Spec version: <x.y> Plan version: <x.y>

### T-<id>-nn – <title>
Satisfies: AC-..., BR-..., EC-...
Depends on: T-...
Parallel-safe with: T-..., T-...
Constraints: plan.md §..., constitution §...
Size estimate: <files> files / <lines> lines
Self-check before handoff:
[ ] <one line per AC, phrased as an observable check>
[ ] No constitution violations
[ ] Nothing implemented that is not traceable to a listed ID
```

A.4 test-spec.md

```
# <FEAT-ID> – Test Specification
Spec version: <x.y>

## 1. Allocation table
| Spec ID | Type | Level | Technique | Tool | Test ID | Gate rung | Owner |

## 2. Oracles
| AC ID | Observable | Observation point | Pass condition | Oracle type |

## 3. Test data strategy
| Dataset | Derived from | Boundary cases | Masking/compliance |

## 4. Environment and mocks
| Dependency | Contract | Mock version | Owner |

## 5. Exploratory charters
| Charter | Target spec weakness | Time box | Tester |

## 6. Exit criteria (the ten items in §15.2, with thresholds)
## 7. Risks and residual exposure
```

A.5 ADR

```
# ADR-<nnn> – <decision title>
Date · Status (Proposed|Accepted|Superseded by ADR-<nnn>) · Deciders

## Context          (the constraint or forcing function)
## Decision         (what we will do, in the active voice)
## Options rejected (each with the reason)
## Consequences     (including the bad ones)
## Constitution impact (deviation? then reference the clause)
```

A.6 Defect record extension

```
Defect: DEF-<nnn>
Classification: CODE | SPEC-GAP | SPEC-WRONG | SPEC-AMBIGUOUS |
               PLAN-VIOLATION | CONTRACT | GENERATION | TEST | ENV/DATA
Spec reference: <AC/BR/EC/NFR ID, or "none – no coverage" for SPEC-GAP>
Classified by:  <human name – never an agent>
Triage sequence step that matched: <1–9 per §19.2>
Resulting artifact change: <spec amendment ref | code PR | generator fix | test fix>
Minimal counterexample: <for property test failures, the shrunk case>
```

APPENDIX B — AGENT PROMPTS

Paste-ready. Each assumes the constitution and relevant spec slice are already in context. Adjust role nouns to your stack.

B.1 Draft a spec from intent (self-spec pattern)

You are drafting a functional specification. Do not write code and do not propose an implementation.

Business intent:
""<paste one paragraph>""

Produce a spec using exactly this structure: Intent; In scope; Out of scope; Actors and preconditions; Behaviour as REQ- items each with one or more AC- items in GIVEN/WHEN/THEN form; Business rules as BR- items; Edge cases and error conditions as EC- items; Non-functional targets as NFR- items; Open questions as OQ- items.

Hard rules:

- Describe WHAT happens, never HOW. No frameworks, class names, table names, SQL, or algorithms.
- Every AC must be mechanically verifiable: a named observable, an observation point, and an exact pass condition.
- Never invent a value I did not give you. Where a number, limit, format, or policy is required and absent, raise it as an OQ- item instead of choosing.
- Use only the domain vocabulary in the intent and the glossary.
- End with a list of every assumption you were tempted to make, and did not.

B.2 Adversarial ambiguity review

Act as an adversarial reviewer whose goal is to misread this specification in defensible ways.

For each finding, output a row:

| Spec ID | Statement | Reading A | Reading B | Which would you implement? | Severity |

Then list separately:

1. Every AC that is NOT mechanically verifiable, with the reason.
2. Every AC with no nameable oracle.
3. Every happy path with no corresponding failure/edge case.
4. Every NFR missing a number, percentile, load condition, or measurement point.
5. Every use of: appropriately, reasonably, gracefully, efficiently, user-friendly, as expected, properly, quickly, robustly, if needed.

Do NOT fix anything. Report only. I will resolve each finding myself.

B.3 Derive acceptance scenarios

Convert this single acceptance criterion into Gherkin.

<paste one AC>

Rules:

- One primary scenario. Tag it @<AC-ID>.
- Use business domain language only. No UI mechanics, no service names, no internal state, no clicks.
- Then produce boundary and negative sibling scenarios implied by the referenced BR-/EC- items, each tagged with the ID it covers.
- Steps must be phrased so the Product Owner can confirm they capture intent.
- Add nothing the spec does not state. If a step needs a value the spec omits, stop and tell me which ID is underspecified.

B.4 Derive property tests from business rules

For each BR- item below, identify whether it expresses an invariant that must hold for every valid input. Candidate invariant shapes: bound, conservation, idempotence, monotonicity, round-trip, ordering, uniqueness.

<paste BR- items>

For each qualifying rule output:

- The invariant as a single precise sentence.
- The property test, using <Hypothesis|jqwik|fast-check>, tagged with the BR- ID.
- The input generator, derived from the schema in contracts/, including nulls, boundaries, maximum lengths, and encoding edge cases.
- One sentence on how you would break the implementation to prove the property actually catches the violation.

For any BR- that is NOT an invariant, say so and say which test level fits instead.

B.5 Implement a task

Implement exactly one task. Nothing more.

Constitution: <inline>

Spec slice: <only the REQ-/AC-/BR-/EC- referenced by the task>

Plan constraints: <only the relevant plan sections>

Task: <paste the task including its self-check list>

Rules:

- Implement only what the listed IDs require. If you believe something else is needed, stop and tell me – do not add it.
- Obey every plan constraint and every constitution clause.
- Write unit tests tagged with the AC/BR IDs they verify.
- Finish by walking the self-check list item by item and stating PASS or FAIL for each, with the file and line that satisfies it.
- If any listed ID is ambiguous, stop before writing code and tell me which.

B.6 Extract a spec from legacy code

Analyse this module and produce a catalogue of its observable behaviours. You are producing a hypothesis, not a specification.

<paste module + any available traffic/log samples>

For each behaviour output:

| Behaviour | Evidence (file/line, log pattern) | Inputs | Observable output |
| Appears intentional / accidental / dead | Confidence | Question for the domain owner |

Rules:

- Describe behaviour as observed. Do not describe the code structure.
- Do not judge whether a behaviour is correct – only whether it appears intended.
- Flag every behaviour that looks like a bug callers may now depend on.
- List behaviours you could not determine without runtime data.

B.7 Drift audit

Compare this specification against this implementation. Report divergence only.

Spec: <paste> Implementation: <paste>

Output three tables:

1. SPECIFIED BUT NOT IMPLEMENTED – spec ID, expected behaviour, evidence of absence.
2. IMPLEMENTED BUT NOT SPECIFIED – behaviour, file/line, why it is not traceable to any spec ID.
3. CONTRADICTS THE SPEC – spec ID, what the spec requires, what the code does.

Do not propose fixes. Do not judge which side is right – that is a human decision under the arbitration rules.

B.8 Spec review for agent-readiness

You are about to implement from this spec. Before doing so, list every decision you would have to make that the spec does not determine for you.

For each: the decision, the options, which you would pick by default, and the spec ID that should have determined it.

Then rate the spec 1–5 for: unambiguity, testability, completeness for your purposes, and scope clarity. Justify each score with a specific example from the text.

Write no code in this response.

APPENDIX C — CI/CD WIRING

Illustrative GitHub Actions; the structure transfers to GitLab CI, Azure Pipelines or Jenkins.

C.1 The gate ladder

```
name: sdd-gates
on: [pull_request]

jobs:
  spec-lint:                                # rung 1
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Validate spec structure and IDs
        run: python tools/spec_lint.py specs/
        # fails on: missing required sections; REQ- with no AC-;
        # malformed IDs; open OQ- with no owner or due date;
        # banned vocabulary (appropriately, reasonably, gracefully, ...)

  contracts:                                # rung 2
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npx @redocly/cli lint specs/**/*.openapi.yaml
      - name: Breaking change check against main
        run: npx oasdiff breaking origin/main:contracts/ ./contracts/ --fail-on ERR

  unit-and-property:                        # rung 3
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pytest tests/unit -q
      - run: pytest tests/property -q --hypothesis-profile=ci # 200 examples on PR

  contract-tests:                           # rung 4
    needs: contracts
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Verify provider against contract
        run: |
          docker run --rm -v "$PWD:/work" -w /work \
            specmatic/specmatic test \
            --testBaseURL=http://localhost:8080

  acceptance:                               # rung 5
```

```

needs: unit-and-property
runs-on: ubuntu-latest
timeout-minutes: 10           # runtime budget is itself a gate
steps:
  - uses: actions/checkout@v4
  - run: behave features/ --tags=~@wip --junit

spec-coverage:                # rung 6
needs: [acceptance, unit-and-property, contract-tests]
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - name: Every approved AC must have a passing linked test
    run: python tools/rtm.py --require-full-coverage --fail-under 100

drift-check:                   # rung 7 (spec-as-source modules only)
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - name: Regenerate and diff
    run: |
      make generate
      git diff --exit-code generated/ \
        || (echo "::error::Generated code was hand-edited. \
          Change the spec and regenerate." && exit 1)

nfr:                           # rung 8 (warn on PR, block on release)
runs-on: ubuntu-latest
continue-on-error: true
steps:
  - uses: actions/checkout@v4
  - run: k6 run --quiet tests/nfr/retry_latency.js
    # thresholds in the script assert NFR- targets directly

publish-rtm:                   # rung 9
needs: spec-coverage
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
  - run: python tools/rtm.py --html --out rtm.html --json --out rtm.json
  - uses: actions/upload-artifact@v4
    with: { name: rtm, path: "rtm.*" }

```

C.2 Spec-drift labelling

```

spec-drift-label:
runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v4
    with: { fetch-depth: 0 }
  - name: Behaviour changed without a spec diff?
    run: |
      SRC=$(git diff --name-only origin/main...HEAD -- src/ | wc -l)
      SPEC=$(git diff --name-only origin/main...HEAD -- specs/ | wc -l)
      if [ "$SRC" -gt 0 ] && [ "$SPEC" -eq 0 ]; then
        gh pr edit "$PR" --add-label spec-drift
        echo "::warning::src changed with no spec change – QE Lead approval required"
      fi

```

Pair this with branch protection requiring the QE Lead as a reviewer on any PR carrying the `spec-drift` label. That single rule does more against specification rot than any amount of process documentation.

C.3 Branch protection for spec files

```
Path: specs/**
Required reviewers: product-owners (1), qe-leads (1)
Required status checks: spec-lint
Dismiss stale approvals on new commits: yes
```

This is the mechanism that gives the QE testability gate in Recipe 14.1 real authority. Do this in week one of the pilot.

APPENDIX D — CHECKLISTS AND RUBRICS

D.1 Spec quality rubric

Score each dimension 0 / 1 / 2. Pass at $\geq 80\%$ of 20 points with **no zero-scored dimension**.

#	Dimension	0	1	2
1	Behaviour focus	Contains frameworks, class names, SQL or algorithms	Some implementation leakage	Pure what, no how
2	Testability	AC requiring human judgement	Some AC lack a named oracle	Every AC has observable, point and pass condition
3	Unambiguity	Weasel terms present	Isolated ambiguity, registered	Two reviewers reach the same reading independently
4	Completeness of edges	No error or edge cases	Some, unsystematic	Failure case for every happy path; EC- items enumerated
5	Scope clarity	No out-of-scope section	Present but vague	Explicit, specific non-goals
6	Identifiability	No IDs	Partial	Every statement addressable
7	NFR precision	Adjectives only	Numbers without conditions	Value + percentile + load + measurement point
8	Business rule separation	Rules buried in prose	Partially extracted	All BR- separately stated and referenced from AC
9	Open-question hygiene	Ambiguity silently resolved	Registered without owners	All OQ- owned and dated
10	Right-sizing	Reads like an LLD, or like a slogan	Uneven	Minimum needed to remove ambiguity, and no more

Dimensions 1 and 10 exist specifically to catch over-specification, which the paper names as the first pitfall.

D.2 Spec review checklist (QE)

- ☐ Every **REQ-** has ≥ 1 **AC-**
- ☐ Every **AC-** has an observable, an observation point, and an exact pass condition
- ☐ Every **AC-** has a nameable oracle
- ☐ Zero banned vocabulary
- ☐ Every happy path has a failure counterpart
- ☐ Every **NFR-** is fully quantified
- ☐ Out-of-scope section present and specific
- ☐ All **OQ-** owned and dated; none affecting in-scope AC left open
- ☐ No implementation detail present
- ☐ Glossary terms consistent with the domain glossary
- ☐ Rubric $\geq 80\%$, no zero-scored dimension

D.3 Agent-output review checklist

- ☐ Every changed line is traceable to a listed AC/BR/EC ID
- ☐ Nothing implemented beyond the task's listed IDs
- ☐ Out-of-scope items from `spec.md` respected
- ☐ All plan constraints respected
- ☐ No constitution violations
- ☐ Unit tests present and tagged with the IDs they verify

- ☐ Tests assert behaviour, not implementation shape
- ☐ Agent's self-check list independently verified, item by item
- ☐ Error handling matches the `EC-` items, not invented
- ☐ No swallowed exceptions, no sleeps in tests, no hardcoded config
- ☐ Reviewer has recorded a spec-satisfaction judgement, not just approval

D.4 Release readiness checklist

- ☐ Spec coverage 100% on in-scope AC
- ☐ Contract verification green for every touched contract
- ☐ Property tests present and passing for every in-scope `BR-` invariant
- ☐ NFR targets measured and met
- ☐ Drift findings: zero
- ☐ All exploratory charters executed; findings classified
- ☐ Zero unclassified defects; zero open Sev 1/2
- ☐ Spec versioned and changelogs if behaviour changed
- ☐ Obsolete scenarios retired and recorded
- ☐ RTM published and archived against the release tag
- ☐ Gate bypasses this cycle: count and reasons recorded

APPENDIX E — RTM GENERATION

E.1 Output shape

REQ	AC	Oracle	Test ID	Level	Tool	Gate	Last result	Defects
REQ-114-01	AC-114-01-a	fixed value	SCEN-114-01-a	Acceptance	Behave	5	PASS	—
REQ-114-01	AC-114-01-b	state check	SCEN-114-01-b	Acceptance	Behave	5	PASS	DEF-2291 (closed)
REQ-114-02	BR-114-01	invariant	PROP-114-01	Property	Hypothesis	3	PASS	—
REQ-114-03	—	contract schema	CT-114-01	Contract	Specmatic	4	PASS	—
REQ-114-04	AC-114-04-a	fixed value	—	—	—	—	NO COVERAGE	—

The final row is what fails the build at rung 6.

E.2 Extractor sketch

```
#!/usr/bin/env python3
"""Join spec IDs to test annotations and emit the RTM. ~80 lines, not a product."""
import re, sys, json, pathlib

ID = re.compile(r'\b(?:REQ|AC|BR|EC|NFR)-[A-Za-z0-9]+-[A-Za-z0-9\-.]+\b')

def spec_ids(root):
    found = {}
    for f in pathlib.Path(root).rglob("spec.md"):
        text = f.read_text(encoding="utf-8")
        approved = "Status: Approved" in text
        for i in set(ID.findall(text)):
            found[i] = {"spec_file": str(f), "approved": approved}
    return found

def test_ids(roots):
    """Collect IDs from Gherkin tags (@AC-...), pytest names, and `covers:` comments."""
    links = {}
    for root in roots:
        for f in pathlib.Path(root).rglob("*"):
            if f.suffix not in {".feature", ".py", ".java", ".ts", ".cs"}:
                continue
            for i in set(ID.findall(f.read_text(encoding="utf-8", errors="ignore"))):
                links.setdefault(i, []).append(str(f))
    return links

def main():
    specs = spec_ids("specs/")
    tests = test_ids(["tests/", "features/"])
    rows, uncovered = [], []
    for sid, meta in sorted(specs.items()):
        linked = tests.get(sid, [])
        rows.append({"id": sid, "tests": linked, **meta})
        if meta["approved"] and not linked and sid.startswith(("AC-", "BR-")):
            uncovered.append(sid)
    pathlib.Path("rtm.json").write_text(json.dumps(rows, indent=2))
    if "--require-full-coverage" in sys.argv and uncovered:
        print("Spec coverage failure - no test links for:", file=sys.stderr)
        for u in uncovered:
            print(f"  {u}", file=sys.stderr)
        sys.exit(1)
    total = sum(1 for r in rows if r["approved"] and r["id"].startswith(("AC-", "BR-")))
    covered = total - len(uncovered)
    print(f"Spec coverage: {covered}/{total}")

if __name__ == "__main__":
    main()
```

Build this in an afternoon. Do not buy it first — build it, learn what you actually need from the RTM, and only then evaluate whether a commercial tool beats eighty lines of Python.

APPENDIX F — GLOSSARY AND SOURCES

F.1 Glossary

Term	Meaning in this cookbook
SDD	Spec-driven development: the spec is the source of truth; code is derived, generated or verified against it
Spec-first	Spec written before code to guide the initial build; may not be maintained afterwards
Spec-anchored	Spec maintained alongside code for the system's life, with alignment enforced by tests
Spec-as-source	Spec is the only human-edited artifact; code is generated and never hand-modified
Vibe coding	Relying on loose prompts, producing inconsistent or erroneous model output
Super-prompt	A spec functioning as structured agent input, decomposing a problem into context-window-sized modules
Self-spec	An agent drafting its own spec from a high-level prompt, for human review before implementation
Constitution	Org-wide non-negotiable engineering rules, loaded into agent context on every task
Drift / specification rot	Divergence between spec and code as code changes and the spec does not
Spec coverage	Share of approved acceptance criteria with at least one passing linked test
Oracle	The source of truth for an expected result: fixed value, schema, invariant, golden file, or stated human judgement
PBT	Property-based testing: verifying invariants across generated inputs rather than fixed examples
Characterisation test	A test pinning current legacy behaviour, written before modification
Orphan code	Code with no traceable requirement behind it
Contract testing	Automated verification that an implementation matches its published interface spec
Arbitration	The decision, on a validation failure, whether the spec or the code is wrong — with the spec as authority
DoR / DoD	Definition of Ready / Definition of Done, restated in spec-satisfaction terms

F.2 Primary source

Deepak Babu Piskala, "*Spec-Driven Development: From Code to Contract in the Age of AI Coding Assistants*," Technical Report, arXiv:2602.00180v1 [cs.SE], 30 January 2026. Sections I–XII. All spectrum definitions, the four-phase loop, the agent-amplification mechanisms, the pitfall set, the tool taxonomy, the three case studies, the decision framework and the Golden Rule are drawn from this paper. Case-study figures (75% integration cycle-time reduction; up to 50% error reduction from human-refined specs) are as reported there; the paper describes the empirical base as nascent.

F.3 Works referenced by the source paper, useful for implementation

Topic	Source
AI-assisted SDD toolkit	GitHub Spec Kit — github.blog, 2025
Practice positioning	Thoughtworks Technology Radar, Vol. 32, 2025
The "not new" critique	B. Finster, <i>5-Minute DevOps: Spec-Driven Development Isn't New</i> , 2025
Practitioner commentary	M. Fowler, <i>Exploring Gen AI: Spec-Driven Development</i> , martinowler.com, 2025
Spec quality and AI output	Red Hat Developer, 2025; InfoQ, Jan 2026
BDD execution	Cucumber docs; Gherkin Reference; Reqroll (formerly SpecFlow); Behave
BDD origin	D. North, <i>Introducing BDD</i> , 2006
TDD	K. Beck, <i>Test Driven Development: By Example</i> , 2002
API contracts	OpenAPI 3.1; GraphQL Specification; AsyncAPI; Protocol Buffers; gRPC
Contract testing	Pact; Specmatic
Design by contract	B. Meyer, <i>Applying Design by Contract</i> , IEEE Computer 25(10), 1992
Model-driven	Mellor & Balcer, <i>Executable UML</i> , 2002; MathWorks Simulink
Safety-critical	ISO 26262, <i>Road vehicles — Functional safety</i> , 2018
AI-assisted SDD tools	Amazon Kiro; Tessl
Code-model evaluation	M. Chen et al., <i>Evaluating Large Language Models Trained on Code</i> , arXiv:2107.03374, 2021

End of cookbook.